

无线电干扰源的快速自动定位与清除模型

摘要

针对有界测向误差下无线电干扰源的快速定位与清除,本文以角楔交会描述位置不确定性,将检测点选择、多源航路调度和定向覆盖逐步结合,建立从单源定位到全域无残留判断的模型与算法。

针对问题一,基于示向度误差有界的特点,本文建立了**有界区间角楔交会模型**,把每个检测点的示向度按误差界张成角楔,干扰源必在各角楔交会的公共部分中。由于形成的定位区域为凸多边形,直径归结为顶点集上的最远点对。使用程序模拟构造的五组算例直径在 10.59 ~ 25.64 m 之间,而以该直径为直径的圆并不总能覆盖定位区域,五组算例中有两组失败,越界顶点到圆心距离超过半径约 10.6%。

针对问题二,只掌握一个检测点处的示向度、源点距离未知,无法先定位再布点,故建立**以期望定位区域直径为准则的布点模型**:由两探测束的交会几何写出定位区域直径,并对源点位置与第二次测向的误差求取期望,得到只依赖第二个检测点相对第一个检测点位置 (a, b) 的二维价值函数 $E[D](a, b)$,使用理论方法求解与数值模拟方法求解获得结果吻合。我们选择一个合理阈值 60m,则**第二检测点和候选区域即为相对位置满足 $E[D] < 60m$ 的区域 $\mathcal{A}_2 = \{(a, b) | E[D](a, b) < 60\}$** ,具体区域形状见下正文。

针对问题三,建立**联合覆盖、定位与清除模型**,以保守定位控制清除风险,以连续覆盖判据验证任务完成,并逐步引入共用航路、沿途定位和逐频道检测调度。在 120 组场景开展五策略配对实验,另作 80 次单因素消融。随机 50 组中,最终的联合调度策略全部清除,完整退出时间均值 3067 s,较初代减少 42.6%。结果表明,联合规划能同时减少行程与检测成本。

针对问题四,建立**连续位置与朝向覆盖模型**,以带空间余量的角区间并集证明无残留,结合保守交会、双侧补测及光学清除覆盖,形成 21 站紧凑布局与信息航路策略。离线 60 组的 779 个源全部清除并认证;随机主集完整退出均值为 5819.1 s,较 25 站联合策略降低 11.25%,完整退出每源均值为 461.3 s。

所建模型以保守几何约束提供定位和退出依据,以期望估价及航路启发式降低成本;其效率受先验分布与候选范围影响,尚无全局最优保证。

关键词: 角楔交会 期望布点 联合航路 连续覆盖

一、问题背景与重述

1.1 问题背景

随着无线电技术的普及和广泛应用，频谱管理中的干扰源定位与清除问题日益突出。干扰不仅影响正常通信，还可能危及公共安全。传统清除方式通常受环境遮挡、低功率发射等影响，精准定位耗时长，且危险环境下人工作业风险高，因此研发机器狗自动定位清除系统代替人工执行任务具有现实需求。但任务中通常面临着目标区域范围较大，干扰源数量、频道、位置、有效接收半径及类型未知，测向存在误差，移动、检测、切换频道和清除过程存在耗时，定向干扰源的发射方向无法辨识等困难。如何在信息不完全且时间受限的条件下高效完成任务，是该自动定位清除系统亟待解决的问题。

1.2 问题重述

考虑到上述实际情况中面临的困难，本文需依次解决以下四个问题：

1. 已知若干检测点的位置及干扰源关于这些检测点的示向度，需建立交会定位区域的几何建模方法，设计计算非空定位区域直径的数值算法，并判定以该直径构成的圆对定位区域的几何覆盖性。
2. 已知在某一个检测点处测得的示向度，需构建第二检测点的空间选择模型与优化策略，确定使定位效果最优的第二检测点候选分布区域。
3. 针对存在多个占用不同频段且总数未知的全向干扰源环境，需设计机器狗从起点出发的自主搜索、交会定位与逐个清除算法策略；通过模拟器演练评估清除成功率与平均定位清除耗时，并完成正式测试。
4. 在问题三的基础上，引入辐射方向未知、数量未知的定向干扰源混合场景，需针对定向辐射特征扩展测向与识别机制，设计兼顾全向与定向干扰源的综合自主检测、定位与清除策略。

二、问题分析

四个子问题层层递进。问题一是单次交会定位的几何基础；问题二则将该几何模型进一步用作检测点位置的选择依据，形成单源定位策略；问题三在源数未知的条件下，把单源定位能力嵌入“测向、收敛、清除”的在线决策；问题四基于问题三模型的基础上，再额外引入定向源方向诊断对决策的影响。

2.1 问题一的分析

本问不宜假设误差分布，而应利用题目给出的误差界，采用最坏情形模型：每个检测点的示向度对应一个由“测量方向 $\pm 1^\circ$ ”张成的角楔。多个检测点的角楔求交，即得干扰源的可能定位区域。由于角楔是凸集，其交仍为凸集（有界时即为凸多边形），因此区域直径可转化为凸多边形顶点间的最远点对问题。求解时，需要先判断交会区域是否有界，再检验以该直径为直径的圆能否覆盖定位区域。整体求解思路可概括为：由误

差界构造单点角楔，通过多检测点角楔交会确定凸多边形定位区域，进而将区域直径计算归结为顶点间的最远点对问题；同时辅以有界性判定与圆覆盖检验。

2.2 问题二的分析

问题的关键在于第二个检测点无法由单次观测唯一确定：只掌握 S_1 处的示向度，源点的距离信息完全缺失，故既不能先定位再布点，也不存在一个位置在每次试验中都最好。本文改以“布点后的定位效果之期望”反向衡量布点位置，沿用问题一中“定位区域直径”这一判据，取其对第二次检测点示向度 θ_2 的测量不确定性与源点位置的不确定性的期望为关键判据，既不依赖某一次定位的运气，也无需先知道源点的距离。又因直径的期望只由两次探测的交会几何决定，它只依赖第二个检测点相对第一个检测点的位置，与绝对坐标无关。于是本问归结为以第二检测点对第一检测点的相对位置为决策变量、以 $E[D](a, b)$ 为价值函数的二维寻优问题，其等值线内部有界域即第二个检测点的候选区域。

2.3 问题三的分析

源数未知使“已发现目标全部清除”并不等于任务完成，因而本问需要同时解决定位清除、未知源搜索和停止判定。对已发现目标，沿用问题一的角楔交会，并结合接收范围与靶区边界构造保守定位区域；再借助问题二的期望直径场，选择有利于缩小区域的补测位置。对尚未发现的频道，则利用无信号观测逐步排除可能藏有源的区域。

两类任务均需移动和检测，若分别规划，容易造成重复绕行。因此，策略先由局部选点扩展为系统覆盖，再将搜索测站与清除目标编入共同航路，进一步根据沿途观测调整测站、访问顺序及逐频道检测时机。为判断何时可以退出，采用覆盖网格并计入单元尺寸余量，将离散检测结果转化为连续区域的排除依据。最终以完整退出时间衡量总成本，以清除比例检验任务完成程度，通过同场景配对比较与机制消融分析各代改进的作用。

2.4 问题四的分析

定向源可能因测点位于发射背面而无信号，因而问题三的距离圆盘排除规则不再适用。本问把未知发射朝向纳入搜索状态，以整方格与连续角区间的覆盖建立无源证书；定位阶段结合角楔外包、短横向基线与光学清除覆盖。算法由格网扫描发展为有证据的联合搜索，再通过压缩站位、改进目标估计和信息排路降低完整退出成本，从而兼顾发现效率与无残留的判断依据。

三、模型假设

为便于建模求解，在合理范围内作如下假设：

1. 示向度误差有界， $|\Delta\theta_i| \leq \varepsilon$ （本题 $\varepsilon = 1^\circ$ ），且同一检测点的误差在观测期间固定不变，只有更换地点才呈现统计规律；故采用有界区间模型（最坏情形），所得区域保证包含真值。
2. 检测点坐标与示向度读数精确已知；示向度以 x 轴正向逆时针为正，取值于 $[0^\circ, 360^\circ)$ 。
3. 干扰源位置在观测与清除期间固定不变，各源频道互不相同且不随时间变化。

4. 测向机与干扰源位于同一平面，区域内无障碍遮挡。
5. 全向源的覆盖角度为 360° ，定向源为定向方向两侧各 90° （含边界），覆盖范围内信号均匀辐射、场强仅随距离衰减。
6. 机器狗沿直线以 5 m/s 匀速行进，忽略转向与加减速耗时；移动中不可检测，单次检测 5 s ，任意两频道切换 1 s 。
7. 与干扰源距离不超过 20 m 时，光学定位（ 3 s ）与激光清除（ 2 s ）必定成功且与覆盖角度无关；距离不超过 5 m 且处于覆盖角度内时无法获得示向度，可直接定位并清除。

四、符号说明

本文使用的主要符号见表 1，未列出的符号在首次出现处说明。

表 1 主要符号说明

符号	含义	单位
$S_i = (x_i, y_i), \theta_i$	第 i 个检测点的坐标与示向度	$\text{m}, ^\circ$
$T = (x_T, y_T), T_c$	单源及频道 c 干扰源的真实位置	m
$\Delta(\alpha, \beta)$	两方位角之差，归一到 $[-180^\circ, 180^\circ]$	$^\circ$
ε, η	示向度误差界及含舍入余量的角楔半宽	$^\circ$
$\mathcal{R}, V$	定位区域及其顶点集	无
D	定位区域直径	m
M	直径圆圆心（直径中点）	m
A, B	直径端点	m
$S_1, S_2 = (a, b)$	第一、第二个检测点及其相对位置	m
$\mathcal{A}_2$	第二个检测点的候选区域	无
$x_{\max}$	干扰源最大可被有效接收距离	m
$E[D](a, b)$	定位区域直径的期望（第二个检测点的价值函数）	m
Ω	干扰源所在的靶区	无
$\mathcal{C}, c$	频道集合及频道索引	无
ρ_c	频道 c 对应干扰源的接收半径	m
$P_c, \tilde{P}_c$	频道 c 的相容定位区域及其保守外包	无
$\hat{x}_c, r_c$	源位置估计中心及外包多边形的包围半径	m
$\mathcal{C}_{\text{unheard}}$	尚未发现干扰源的频道集合	无
h, δ_h	认证方格边长及半对角线 $h/\sqrt{2}$	m
$\gamma, d(\gamma)$	定向源的发射朝向角及单位方向向量	rad , 无
$I_s(q), Q_c^-$	格中心 q 的可排除朝向弧及频道负观测站集	无
$N_{\text{clear,ok}}$	成功清除次数（亦为成功清除源数）	无
L	机器人完成任务的总行程	m
$T_{\text{exit}}, T_{\text{last}}$	完整退出时间与末次成功清除时刻	s

五、模型建立与求解

5.1 问题一：示向度交会定位模型

5.1.1 模型建立

设第 i 个检测点为 $S_i = (x_i, y_i)$, 测得的示向度为 θ_i (x 轴正向逆时针, 取值 $[0^\circ, 360^\circ)$), 误差界为 $\varepsilon = 1^\circ$ 。干扰源则落在以 S_i 为顶点、关于示向度方向对称张开 2ε 的角楔之内 (为避免方位角相减在跨 0° 时出错, 记两角之差 $\Delta(\alpha, \beta) = ((\alpha - \beta + 180^\circ) \bmod 360^\circ) - 180^\circ$):

$$\mathcal{W}_i = \{S_i\} \cup \{T \neq S_i : |\Delta(\beta(S_i, T), \theta_i)| \leq \varepsilon\}, \quad (1)$$

其中 $\beta(S, T)$ 为自检测点 S 指向干扰源 T 的方位角。 m 次测向给出的定位区域即各角楔之交, 如图 1 所示 (以 $m = 3$ 为例)。每个角楔可表示为两个闭半平面之交; 各角楔对干扰源的约束同时成立, 故定位区域为 $\mathcal{R} = \bigcap_{i=1}^m \mathcal{W}_i$, 它是 $2m$ 个半平面之交, 为凸多边形区域, 可能为空、无界, 或退化为点、线段。非空有界时其顶点数不超过 $2m$, 且每个顶点必是某两条边界直线的交点。记该区域的顶点集为 V 。

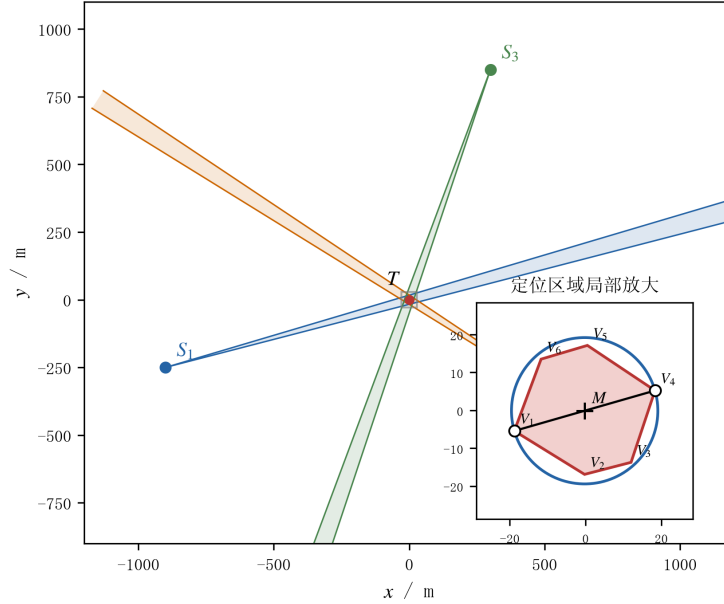


图 1 多检测点示向度交会得到的定位区域; 右下角的局部放大图中标出交点 V_i 和直径中点 M

若示向度方向过于接近, 非空定位区域 $\mathcal{R}$ 将沿公共方向无限延伸, 其直径为 $+\infty$, 故在确认非空后必须判定其有界性。从几何直观来看, 定位区域无界是无穷远处方向一致性导致的拓扑性质, 其判据仅取决于各探测点的方向角范围。将各探测点 S_i 视作顶点、测向误差张角区间 $\Theta_i = [\theta_i - \varepsilon, \theta_i + \varepsilon]$ 视作无限延伸的闭角楔, 区域无界当且仅当所有角楔的方向区间存在公共交集, 证明如下: 由于公共交集非空, 从区域内任意基准点出发, 沿该公共方向平移形成的射线将永远不会离开任何一个角楔, 导致区域沿该方向伸展至无穷远; 反之, 若区域无界, 则必存在趋于无穷远的点列, 由单位圆周的紧性可知, 自各探测点指向该点列的方位角必定收敛于某个极限方向, 又因角楔区间为闭集, 该极限方向必落在每个角楔的方向区间内, 从而保证了公共交集的存在。特别地,

当 $m = 2$ 时，定位区域无界当且仅当两次测向的方向夹角不超过 2ε 。该判定逻辑无需依赖各检测点的具体空间坐标，仅需进行 $\mathcal{O}(m)$ 次简单的区间重叠比较即可快速完成。

对已判定为有界的区域，凸多边形上任意两点的最大距离必由两个顶点达到。故求直径不必在连续区域上作最优化，只需在顶点集 V 上比较点对距离：

$$D = \max_{P, Q \in \mathcal{R}} \|P - Q\| = \max_{P, Q \in V} \|P - Q\|. \quad (2)$$

设使式 (2) 取到最大值的两个顶点为 A, B ，则直径圆圆心为 $M = (A + B)/2$ 、半径为 $r = D/2$ 。

覆盖与否由圆盘的凸性直接判定： $\mathcal{R} \subseteq$ 该直径圆当且仅当其全部顶点含于圆内，即

$$\max_{P \in V} \|P - M\| \leq D/2. \quad (3)$$

条件不满足时，必有顶点 P 使 $\angle APB < 90^\circ$ 而落在圆外。注意该直径圆的圆心固定为 M ，与圆心可自由选取的最小包围圆不同，故式 (3) 需逐个顶点检验。注意到最远点对可能不唯一，但覆盖判定与取法无关：若某一对最远点 A, B 确定的圆已覆盖 $\mathcal{R}$ ，则 $\mathcal{R}$ 含于半径 $D/2$ 的圆内，而 $\mathcal{R}$ 内任意两点距离不超过 D ，故任何其他最远点对都必对应于该圆上的一条直径，其确定的圆与原圆重合；若某一对不能覆盖，则任何一对也都不能。因此下文任取一对最远点构造直径圆即可。

5.1.2 模型求解算法

算法输入为 m 组“检测点坐标与示向度”及误差界 ε ，输出为顶点集 V 、直径 D 、直径端点 A, B 、圆心 M 及覆盖判定，流程如图 2，分四步。

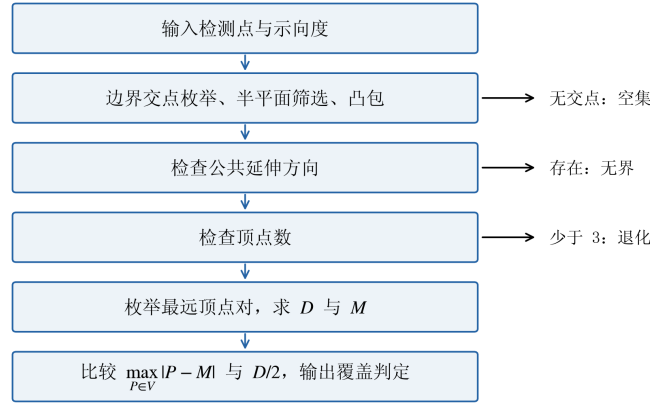


图 2 问题一求解算法流程

第一步（区域构造与非空检验）：区域的每个顶点都是两条边界直线的交点，故算法枚举每个角楔两条边界射线所在直线（共 $2m$ 条）的两两交点，再用全部 $2m$ 个半平面判据逐一筛选；同一角楔两条边界线的交点即检测点 S_i ，它同样作为候选点保留。随后对候选点集求凸包（包含全部候选点的最小凸多边形，用 Andrew 单调链实现），剔除重复点与近似共线点，得到逆时针排列的顶点表 V 。若候选点集为空，说明各角楔无公共交集（测量数据自相矛盾），算法返回状态 `empty`（无可行定位区域）并终止该例计算。

第二步（有界性与退化检验）：对非空交集，检查 m 个方位角区间 $[\theta_i - \varepsilon, \theta_i + \varepsilon]$ 是否存在公共方向。若存在，返回 **unbounded**；若不存在且 $|V| < 3$ ，返回 **degenerate**，表示点或线段。其余情形得到有界凸多边形，继续计算直径。

第三步（求直径）：按式 (2) 枚举 V 中的顶点对取最大距离，对应两点记为 A, B ，圆心取 $M = (A + B)/2$ ；对本文算例规模直接枚举即可，计算量为 $O(|V|^2)$ 。程序在顶点较多时使用旋转卡壳算法，其距离比较次数随顶点数线性增长；另以两法交叉计算检查直径的一致性。区域构造需枚举 $O(m^2)$ 个交点，每个交点检验 $2m$ 个约束，故直接实现的总体计算量为 $O(m^3)$ 。

第四步（覆盖判定）：按式 (3) 计算 $\max_{P \in V} \|P - M\|$ ，判定条件为

$$\max_{P \in V} \|P - M\| \leq D/2 + \tau, \quad (4)$$

其中绝对容差 $\tau = 10^{-9} \text{ m}$ 用于容忍相切构型的浮点误差，避免相切构型被误判为不覆盖；判定不覆盖时，输出取到该最大值的顶点作为越界位置。

为保证结果不依赖具体实现，程序另有一套独立等价的区域构造算法：自一个大正方形出发，用 Sutherland–Hodgman 算法逐个半平面裁剪多边形。在五组算例上的顶点最大偏差小于 10^{-9} m ，直径枚举与旋转卡壳的结果一致，状态名与原因说明见附录 B 的 `analyse()`。

5.1.3 求解结果与分析

使用程序构造五组良态算例（检测点分布在干扰源四周，方位张角 $180^\circ \sim 249^\circ$ 、最小方位间隔不小于 25° 、与干扰源距离 $160 \sim 1300 \text{ m}$ ），结果见表 2；五组算例的定位区域直径均在 $10.59 \sim 25.64 \text{ m}$ 之间，即两次以上交会已能把候选范围从测向距离的百米量级收缩到数十米以内。表中同时给出覆盖判定，其两种结果的对照见图 3：左图（算例 A_1 ）的定位区域全部顶点都落在直径圆内或圆周上，最远顶点到圆心 M 的距离恰为 $D/2$ （比值 1.0000），此时直径圆就是该区域的最小包围圆；右图（算例 A_2 ）中，橙色虚线标出的顶点 P 落在圆外， $|PM| = 10.29 \text{ m}$ 比 $D/2 = 9.31 \text{ m}$ 大 0.98 m （比值 1.1055），直径圆覆盖判定为“否”。

表 2 问题一算例的计算结果

算例	m	顶点数	$D \text{ (m)}$	$D/2 \text{ (m)}$	$\max \ P - M\ \text{ (m)}$	比值	覆盖
A_1	6	6	17.64	8.82	8.82	1.0000	是
A_2	5	5	18.61	9.31	10.29	1.1055	否
A_3	4	5	25.64	12.82	13.75	1.0722	否
A_4	5	4	21.12	10.56	10.56	1.0000	是
A_5	6	5	10.59	5.30	5.30	1.0000	是

可见五组算例中有两组（ A_2 、 A_3 ）的直径圆无法覆盖定位区域，越界顶点到 M 的距离分别比 $D/2$ 大 0.98 m 与 0.93 m ，超出 10.55% 与 7.22%；这两组算例的区域均为斜五边形，由于 A_2 中两个顶点距离较近，故可以近似为四边形，而越界顶点所对直径形成的张角小于 90° （图 3 右图中圈出的顶点张角为 84.2° ），因而落在圆外。

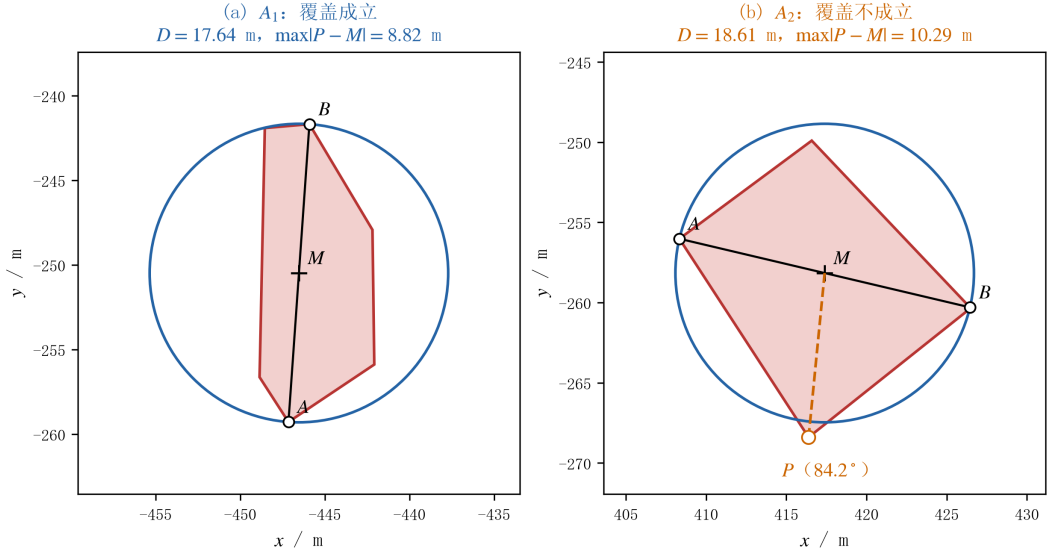


图3 直径圆覆盖判定的两种结果：(a) 覆盖成立；(b) 覆盖不成立

表3 四组随机扫描的覆盖失败统计（固定种子，理想/实际构型逐例配对）

构型	检测点数	抽样数	有界有效数	失败数	失败率	最坏比值	需放大约
理想	2	20000	20000	231	1.16%	1.0336	3.4%
理想	3 ~ 6	4000	3472	113	3.25%	1.0304	3.0%
实际	2	20000	19996	246	1.23%	1.0346	3.5%
实际	3 ~ 6	4000	3472	993	28.60%	1.6876	68.8%

注：理想构型指各示向度精确指向源点；实际构型指在同一批构型上叠加与算例生成同分布的 $\pm\epsilon$ 测向误差；失败率以有界有效数为分母，“需放大约”为最坏比值减1。两组3 ~ 6点扫描各有528组未满足方位间隔要求而被筛除，不计入有界有效数。

为检验覆盖失败是否仅为个别现象，本文以固定随机种子作四组几何扫描（表3）。叠加测向误差后，3 ~ 6个检测点的覆盖失败率由3.25%升至28.60%、最坏时直径圆需放大约69%。测向误差可能改变多边形形状并提高覆盖失败比例，但误差不一致并不必然导致失败。上述比例针对本文规定的构型采样分布，不能解释为所有实际布点的普遍失败概率。其中两点扫描用于检验纯角楔几何性质，未按有效接收半径筛选检测点。本节数值的数据清单与复现命令见附录A。

5.2 问题二：第二检测点的选择模型

5.2.1 模型建立

记号与概率描述

S_1 置于原点，其示向度方向取为 x 轴正方向； S_2 置于 (a, b) 。已知 S_1 的示向度后，源点 T 可能位置 (x_T, y_T) 服从一个条件概率分布

$$T \sim P_T(T), \quad (5)$$

而给定源点位置与 S_2 的位置， θ_2 的读数由于误差亦不能确定，近似服从

$$\theta_2 \sim P_\theta(\theta_2 | T, S_2). \quad (6)$$

位形参数 (θ_2, a, b) 一经确定，定位区域直径便由几何关系唯一确定给出：

$$D = D(\theta_2, a, b). \quad (7)$$

于是 D 服从由几何参量 (a, b) 控制的概率分布，第二个检测点的选择问题即化为求如下二维价值函数：

$$E[D](a, b) = \iint d^2 \vec{r}_T P_T(T | S_1) \int d\theta_2 D(\theta_2, a, b) P_\theta(\theta_2 | T, S_2), \quad (8)$$

$E[D](a, b)$ 越小，在该点放置第二个检测点所得定位平均效果越好。这即为问题二求解的关键衡量指标。

5.2.2 理论求解

(a) 理论推导

式 (8) 含内外两次共计三重概率积分，直接使用不便，故作以下五条近似：

1. S_2 距 S_1 的距离 R 不太小。该近似的合理性并非天然成立： S_2 距 S_1 过近时两视线的交角趋于零，所得定位区域必然很大，这是我们不希望的，故 R 较小的区域本来就不在选择范围之内，而本问关心的待选区域附近该近似较好。
2. 忽略源点纵坐标，认为其恰好落在 x 轴上，其位置只由一个参数 x_T 描述。当 R 不小时源点纵坐标 y_T 是小量，将分布对 y_T 作展开：

$$P_\theta(\theta_2 | x_T, y_T, a, b) \sim P_\theta(\theta_2 | x_T, 0, a, b) + \frac{d}{dy_T} P_\theta \Big|_{y_T=0} y_T + o(y_T) \quad (9)$$

而在第一个概率积分中区间关于 $y_T = 0$ 对称，一阶项相互抵消，故 y_T 对 $E[D]$ 的影响是高阶小量。

3. 考虑 y_T 小量近似后所得零阶项 $P_\theta(\theta_2 | x_T, 0, a, b)$ 在扇形区间内积分，可认为先验权重满足 $P_T(x_T | S_1) \propto x_T$ ，归一化后得

$$P_T(x_T | S_1) = \frac{2x_T}{x_{max}^2}, \quad x_T \in [0, x_{max}], \quad (10)$$

其中 x_{max} 为干扰源的最大可被有效接收距离，在本问中可直接取为 1500m。

4. 与第 2 条同理，由于检测点二实际测得方向角在理论值 $\theta_{2,ideal}$ 左右 1° 区间内均匀分布，偏离值为小量认为其分布近似 $\theta_2 \sim \delta(\theta_2 - \theta_{2,ideal})$ ，即等效直接认为第二次探测所得方位正对源点。
5. 在计算第二检测点的定位区域直径期望时忽略探测半径的影响，而将第二检测点的定位区域角楔延伸到足够长进行计算。这条近似在第二检测点距离第一检测点定位区域不太远时成立。因为此时整个第一检测点的定位区域完全包含在第二检测点的定位区域内，第二检测点的探测半径对定位区域直径的影响可以忽略。

五条近似把价值函数精简为单个一维积分：

$$E[D](a, b) = \int_0^{x_{max}} dx_T P_T(x_T | S_1) D(x_T, a, b). \quad (11)$$

(b) 定位区域直径的解析推导

近似取定 θ_2 正对源点后，两条探测视线在源点处的交角 α 满足

$$\sin \alpha = \frac{|b|}{r_{2T}}, \quad r_{2T} = \sqrt{(x_T - a)^2 + b^2}, \quad (12)$$

图 4 为一个特定情况下的两个检测点示意图，并取源点 T 、第二检测点 S_2 ；图中蓝、黄两个区域分别为两次探测各自的定位区域， $\angle S_1TS_2 = \alpha$ 为两视线在源点处的交角，右上角为两区域相交所得的定位区域 $ABCD$ 的局部放大图。所得定位区域为局部图中的红色近似平行四边形 $ABCD$ ：该区域的直径即对角线 $|BD|$ 。

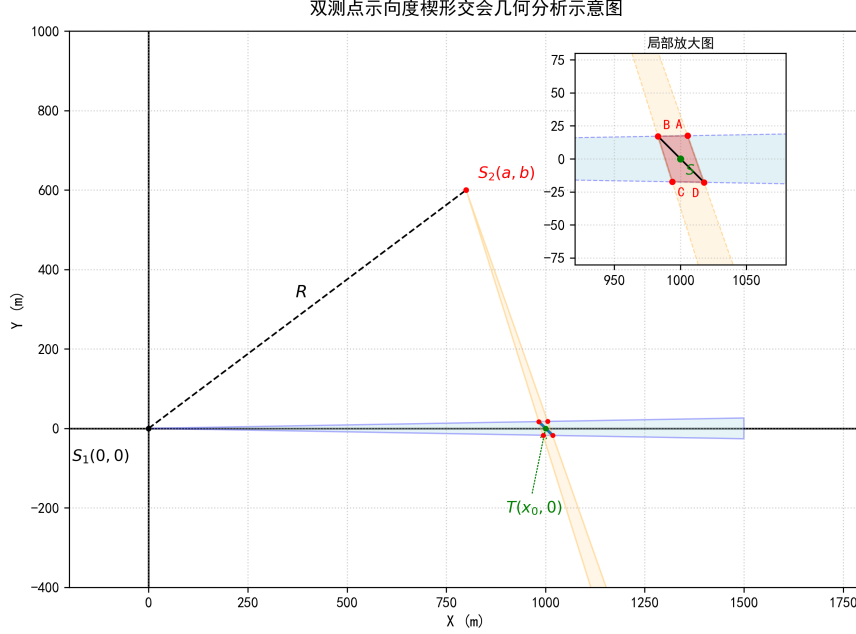


图 4 两探测束交会产生定位区域示意图

本文探测束张角极小 ($\Delta\theta = 2\varepsilon = 2^\circ \approx 0.0349 \text{ rad} \ll 1$)，故在源点处两次探测束的横向宽度分别为

$$w_1 = x_T \Delta\theta, \quad w_2 = r_{2T} \Delta\theta = \sqrt{(x_T - a)^2 + b^2} \Delta\theta. \quad (13)$$

由式 (12) 及 $(x_T - a)/r_{2T}$ 的符号，交角还满足

$$\cos \alpha = \frac{x_T - a}{r_{2T}} = \frac{x_T - a}{\sqrt{(x_T - a)^2 + b^2}}. \quad (14)$$

其对角线长度由两组对边的宽度与夹角 α 完全确定：

$$D(x_T, a, b) = \frac{\sqrt{w_1^2 + w_2^2 + 2w_1w_2 \cos \alpha}}{\sin \alpha}. \quad (15)$$

该式有直接的几何解释：下标 1、2 的两项分别对应两次探测各自的束宽，交叉项 $2w_1w_2 \cos \alpha$ 则来自两组对边之间的夹角。

把式 (13)、(12) 与 (14) 代入式 (15)，即可消去 w_1, w_2, α ，将直径完全显式化为 a, b, x_T 的函数。分子先提出公因子 $r_{2T}\Delta\theta$ ，被开方式化为 $x_T^2 + r_{2T}^2 + 2x_T|x_T - a|$ ，于是

$$\begin{aligned} D(x_T, a, b) &= \frac{\Delta\theta r_{2T}}{|b|} \sqrt{x_T^2 + r_{2T}^2 + 2x_T(x_T - a)} \\ &= \frac{\Delta\theta}{|b|} \sqrt{(x_T - a)^2 + b^2} \sqrt{(x_T + |x_T - a|)^2 + b^2}, \end{aligned} \quad (16)$$

其中第一个根式即式 (12) 中的 $r_{2T} = |S_2T|$ ，第二个根式是源点到点 $(a/2, -b/2)$ 之距离的两倍。定位区域直径由这两个几何量的乘积与 $|b|$ 之比给出。

式 (16) 的合理性可由两个极限情形检验：当第二个检测点正落在第一个视线方向附近 ($a \rightarrow x_T, b \rightarrow 0$) 时交角趋于 90° ，此时 $D \rightarrow \Delta\theta x_T = w_1$ ，恰为定位区域沿垂向的宽度；当 $b \rightarrow 0$ 且 $x_T \neq a$ 时两视线共线、 $\sin \alpha \rightarrow 0$ ， $D \rightarrow \infty$ ，与式 (12) 的退化判据一致。

代入 $\Delta\theta = 2^\circ = \pi/90 \text{ rad}$ ，得直径函数的解析表达式

$$D(x_T, a, b) = \frac{\pi}{90|b|} \sqrt{(x_T - a)^2 + b^2} \sqrt{(x_T + |x_T - a|)^2 + b^2}. \quad (17)$$

再代回式 (11) 并利用式 (10)，即得可直接用于数值积分与优化的目标函数

$$E[D](a, b) = \frac{\pi}{45 x_{max}^2 |b|} \int_0^{x_{max}} x_T \sqrt{[(x_T - a)^2 + b^2][(x_T + |x_T - a|)^2 + b^2]} dx_T. \quad (18)$$

引入无量纲变量 $u = x_T/x_{max} \in [0, 1]$ 与 $\hat{a} = a/x_{max}$ 、 $\hat{b} = b/x_{max}$ ，式 (18) 还可写成

$$E[D](a, b) = \frac{\pi x_{max}}{45 |\hat{b}|} \int_0^1 u \sqrt{[(u - \hat{a})^2 + \hat{b}^2][(u + |u - \hat{a}|)^2 + \hat{b}^2]} du, \quad (19)$$

即 $E[D]$ 只依赖 x_{max} 与无量纲位形 $(\hat{a}, \hat{b})$ ，与绝对尺度无关。

(c) 价值函数的数值求积

价值函数的不定积分核为

$$\int x_T \sqrt{[(x_T - a)^2 + b^2][(x_T + |x_T - a|)^2 + b^2]} dx_T. \quad (20)$$

由于 $|x_T - a|$ 在 $x_T = a$ 处分段，式 (20) 需在 $[0, x_{max}]$ 内按 $x_T < a$ 与 $x_T \geq a$ 分段处理；本文直接采用中点求积，避免对分段根式寻求原函数。该数值积分与理论闭式直径公式保持一致。

我们使用 Mathematica 软件对理论公式 (18) 进行了计算绘图，计算结果见 5.2.4 节。

5.2.3 数值方法求解

为了与由理论近似解析式得到的 $E[D](a, b)$ 交叉验证，我们使用 Python 程序对该分布作了数值求解。算法输入为已知示向度方向（用于建立坐标系）、有效接收半径 x_{max} 与误差界 ε ；输出为定位区域直径期望的分布 $E[D](a, b)$ 最优相对位形 (a^*, b^*) 、最小期望直径 $E[D]_{\min}$ 与候选区域，分四步。

第一步建立标准坐标系： S_1 置于原点，其示向度方向取为 x 轴正向， $S_2 = (a, b)$ ；于是几何量 $|b|$ 、 r_{2T} 、 α 都可用 a, b, x_T 写出。

第二步 D 的精确计算：对给定 (a, b, x_T) ，作 S_1 的两条边界射线（与 x 轴成 $\pm\varepsilon$ ）与 S_2 的两条边界射线（绕“ $S_2 \rightarrow T$ ”方向成 $\pm\varepsilon$ ）。四条直线两两求交得 6 个候选点，再用角楔定义筛出真正落在两个楔形内的顶点（良态下恰为 4 个，即四边形 $ABCD$ ），取

$$D(x_T, a, b) = \max_{P, Q \in \{A, B, C, D\}} |PQ|; \quad (21)$$

两条边界线平行（两视线共线）时交会区域无界，该源位样本直接舍弃。这一步用的是四边形的严格几何，与前文解析式的平行四边形近似无关。

第三步求期望。源位采样取在 x 轴正半轴上，上限为有效接收半径上界，在 $[0, x_{max}]$ 上取 n 个分点，以 $w_k \propto x_T^{(k)}$ （即式 (10)）加权。要注意只有“第二个检测点也能收到

该频道信号”的源位样本才是可用的：对每个样本还须满足 $|S_2 G_k| \leq x_{max}$ 。把不满足该条件或无界的样本剔除后，在**剩余样本**上重新归一，得

$$E[D](a, b) = \frac{\sum_{k \in \mathcal{K}(a, b)} w_k D(x_T^{(k)}, a, b)}{\sum_{k \in \mathcal{K}(a, b)} w_k}, \quad w_k \propto x_T^{(k)}, \quad (22)$$

其中 $\mathcal{K}(a, b)$ 为可用样本集合。式 (22) 是条件期望 $E[D | \text{两次探测均有效}]$ ，而非对所有源位的无条件平均；这一区别在 S_2 远离 S_1 时不可忽略，故本文一律采用式 (22)。

第四步寻优：网格取以 x_{max} 为半径的靶区圆域、步长 30 m，剔除 $|y| \leq 60$ m 的近轴退化带（该带内两视线近简并、交会区域被拉成极窄长条甚至无界），其余格点逐点代入式 (22)，取最小值点即 (a^*, b^*) 。为避免近轴期望值发散区域干扰绘图颜色映射，程序中把 $E[D]$ 超过全体可用格点中位数 10 倍的格点标注为不可用带，不参与寻优。

5.2.4 求解结果与分析

我们取 $x_{max} = 1500$ m、 $\varepsilon = 1^\circ$ ，通过上述两套相互独立的计算方法分别得到结果，并在最后据此确定第二检测点的待选区域。

(a) 理论解计算结果

按照 5.2.2 节分析，我们使用 Mathematica 软件进行计算，将式 (19) 的闭式被积函数，把式 (17) 的直径代入对源位的加权积分，在 15 m 步长的矩形网格上采用 200 个中点进行加权求积，计算网格覆盖 $a \in [-500, 1795]$ m、 $b \in [-1800, 1800]$ m。图 5(a) 给出 $E[D](a, b)$ 在所在区域内的函数图像。其中 $a < -500$ 区域不在计算处理范围内，绘制为白色。而接近 x 轴部分 $E[D]$ 发散，超出绘图范围与机器精度限制，亦绘制为白色。理论网格上的最小值为 $E[D]_{\min} = 57.214$ m，位于 $(1060, \pm 645)$ m。

(b) 数值模拟计算结果

我们按 5.2.3 节的算法，使用 Python 程序，对每个格点由两个角楔的严格几何求出交会四边形直径，再对源位加权求积绘制了 $E[D](a, b)$ 图像；网格为以 x_{max} 为半径的靶区圆域、步长 30 m。由于在数值模拟中并未采取理论推导时的近似，故绘图区域限制更大。 $|S_2 S_1| > x_{max}$ 或 $|S_2 G| > x_{max}$ 导致两次探测不可能同时有效接收的不可行区，与两视线近似共线、 $E[D]$ 超出色标上限而被截断的区域均绘制为白色

图 5(b) 给出同一价值函数由数值模拟得到的分布。与 (a) 相比，两者的整体形态、极小值位置与色带走向一致，极小值为 $E[D]_{\min} = 57.34$ m，位于 $(1080, \pm 660)$ m。

(c) 结果交叉对比

图 5 中的两幅图绘制在参数与颜色映射完全相同的两个坐标系中。直观可见在二者定义域重叠范围内，计算结果的整体形态吻合度极高。

为进一步定量分析，我们把 MMA 的 15 m 网格结果双线性插值到数值模拟的 30 m 格点上，对两者直接逐点比对。表 4 列出若干代表性位置的结果。在最优带 ($E[D] \leq 60$ m 区域) 内 308 个可对比点上，两者的绝对相对偏差中位数为 0.16%、最大为 0.32%；放宽到 $E[D] \leq 100$ m 的 3296 个点上，绝对相对偏差中位数为 0.24%、最大为 0.98%。两条独立的求解路线在关心的取值范围内保持高度一致，支持 5.2.2 节解析推导与 5.2.3 节四边形几何实现的数值一致性。

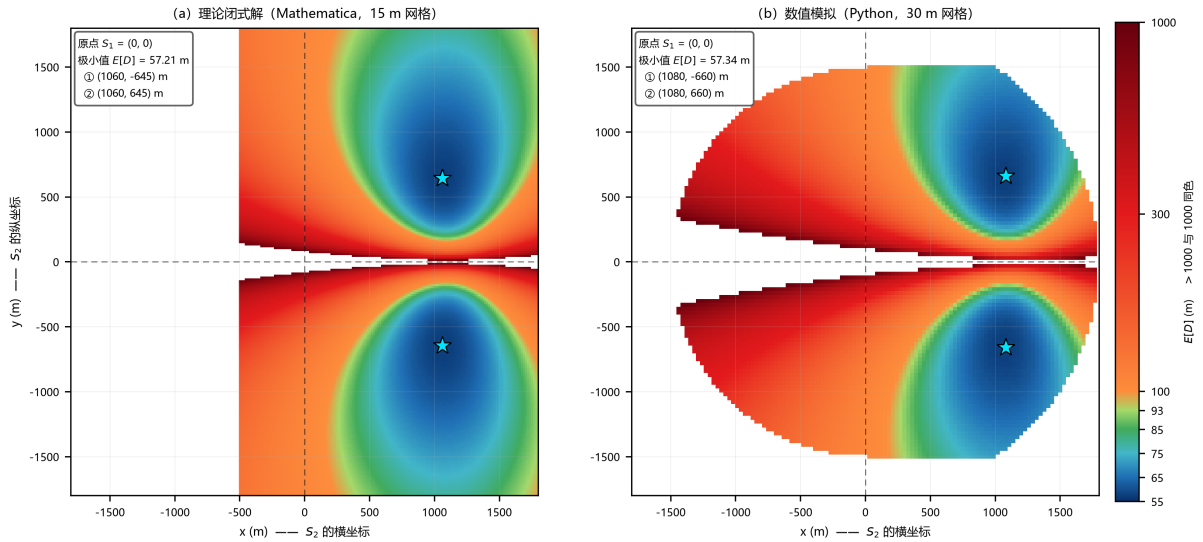


图 5 期望定位区域直径的分布：理论闭式解与数值模拟的并列对比。(a) Mathematica 按解析式在 15 m 网格上求积；(b) Python 按交会四边形严格几何在 30 m 网格上求解。两图的极小值位置与色带走向一致

表 4 理论闭式解与数值模拟在代表性第二检测点位置的对比

a (m)	b (m)	理论闭式解 $E[D]$ (m)	数值模拟 $E[D]$ (m)	相对偏差
1080	660	57.244	57.336	+0.16%
1050	660	57.245	57.351	+0.19%
1110	750	57.776	57.849	+0.13%
1200	690	58.930	58.963	+0.06%
1200	600	59.191	59.222	+0.05%
1200	900	60.439	60.474	+0.06%
900	900	61.013	61.154	+0.23%
1350	750	64.439	64.406	-0.05%
900	1200	65.951	66.066	+0.17%
1410	600	69.947	69.856	-0.13%
1500	900	72.769	72.684	-0.12%
600	600	78.090	78.498	+0.52%
300	900	94.150	94.560	+0.44%

在两种计算结果中观察 $E[D]$ 图像性质，其取值从 57 m 一直延伸到色标上限 10^3 m 以上，且在极小值附近平缓而在 x 轴附近发散，故图中采用非线性色标：把 55 ~ 100 m 一段拉开、100 m 以上压缩，提升问题更关心的低值区的区分度。图像关于 $b = 0$ (S_1 的视线方向) 严格上下对称，极小值成对出现，这是由体系对称性所决定的。 $E[D] \leq 60m$ 的最优带为极小点周围的一小片区域，在此范围内靠近和远离检测点一的探测区间，都会使得最终探测区间的直径期望增大。这是由于当检测点二距离检测点一的定位区域

很近时，观测角夹角正弦 $\sin\alpha$ 平均值小，使得最终定位区域重叠极大，直径快速发散。当检测点二距离检测点一的定位区域很远时，检测点二的定位区域在干扰源点处展宽已经变得较大，使得定位区域直径增大。

(d) 待选区域确定

问题二要求给出的是第二检测点的可选范围。综合上文计算结果，本文**选取 $E[D] \leq 60$ m 的区域作为第二检测点的待选区域**：该阈值比极小值高约 5%，既保证定位效果（期望直径接近销毁半径数量级），又给现场留出足够的位形余量。具体范围见图 6 所示。两套计算给出的同一区域的边界几乎重合。实际布点时建议取区域中心附近的 $(a, b) \approx (0.71 x_{max}, 0.43 x_{max}) = (1060, 645)$ m，此时 $E[D]$ 接近极小值，在期望意义下最利于确认干扰源位置。

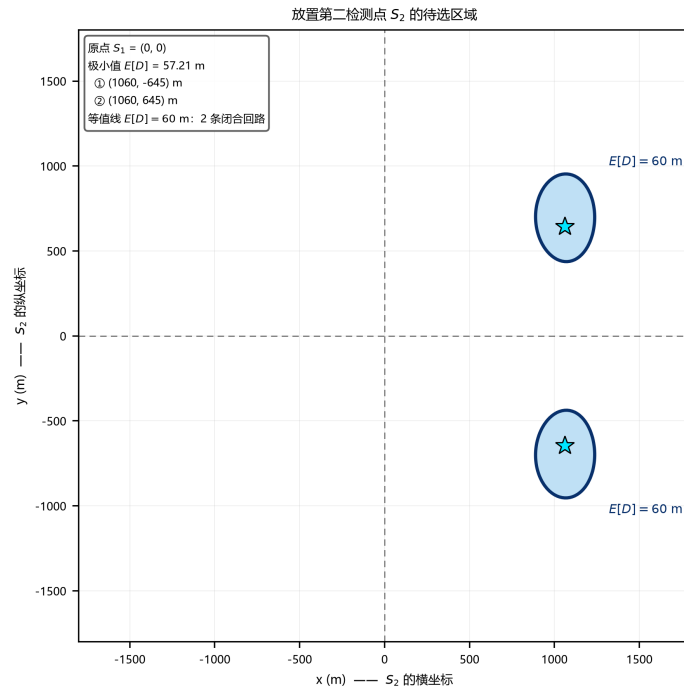


图 6 $E[D] \leq 60$ m 等值线围出的第二检测点待选区域

5.3 问题三：多源定位与清除策略

5.3.1 模型建立

问题三的源数未知，而每个频道至多对应一个静止全向源，因此，完成任务不仅要求清除已发现目标，还须排除其余频道存在残留源的可能。为此，先建立统一时间成本，再分别描述目标定位与区域排除的约束。

(a) 任务成本与评价指标

设靶区 $\Omega = \{x : \|x\| \leq 1800\}$ ，频道集合 $\mathcal{C} = \{1, \dots, 20\}$ ，源接收半径 $\rho_c \in [1000, 1500]$ m。机器人从原点出发，速度为 5 m/s；测量耗时 5 s，切频耗时 1 s。清除需先进行 3 s 光学定位，成功时另需 2 s。令 $L = \sum_k \|s_k - s_{k-1}\|$ 为相邻动作终点间的总行程， N 为对应动作次数，则完整退出时间为

$$T_{\text{exit}} = \frac{L}{5} + 5N_{\text{measure}} + N_{\text{switch}} + 5N_{\text{clear,ok}} + 3N_{\text{clear,fail}}. \quad (23)$$

测向机初始频道为 1，清除不改变频道；进入和退出操作不增加上述时间。以 T_{exit} 为主要指标，同时报告清除比例及各次实验的 $T_{\text{exit}}/N_{\text{clear,ok}}$ ，避免将漏清造成的提前退出视为效率提高。末次清除时刻记为 T_{last} ，两者之差表示验证其余频道所需的时间。

(b) 观测约束与保守定位区域

将收到示向度的测量称为正观测，无信号测量称为负观测；近场观测则表明源距测点不超过 5 m，可原地清除并确认结果。正观测用于定位，负观测用于排除，两类信息共同构成逐频道状态。

沿用问题一的角楔交会方法，令 $u(\theta) = (\cos \theta, \sin \theta)$ ， $\text{cross}(a, b) = a_x b_y - a_y b_x$ 。对测站 S_i 和示向度 θ_i ，取含舍入余量的误差半宽 $\eta = (1.005 + 10^{-9})^\circ$ ，得到

$$W_i = \{x : \text{cross}(u(\theta_i - \eta), x - S_i) \geq 0, \text{cross}(u(\theta_i + \eta), x - S_i) \leq 0\}. \quad (24)$$

记 $B(s, r)$ 为以 s 为中心、半径为 r 的闭圆盘， I_c^+ 为频道 c 的正观测索引。结合靶区及接收范围，有

$$P_c = \Omega \cap \bigcap_{i \in I_c^+} (B(S_i, 1500) \cap W_i) \subseteq \tilde{P}_c. \quad (25)$$

其中 $\tilde{P}_c$ 是包含所有相容源位的保守外包：以每圆 32 条外切半平面代替圆域，再与角楔求交。相对估计中心 $\hat{x}_c$ ，令 r_c 为其到外包多边形各顶点的最大距离，即有 $P_c \subseteq B(\hat{x}_c, r_c)$ 。当 $r_c \leq 20$ m 时，到中心清除具有保证；否则仍须补测或试探。负观测可修正点估计，但不用于缩小这一凸外包。

(c) 连续覆盖与任务完成判据

定位解决已发现目标的位置不确定性；对未发现频道，还需判断是否存在未被检测的区域。用边长 h 的闭方格覆盖靶区，保留所有与圆域相交的单元，包括中心位于圆外的边界单元。单元内点到中心 q 的距离至多为 $h/\sqrt{2}$ ，故取

$$r_{\text{cert}} = 1000 - \frac{h}{\sqrt{2}} - \delta_r, \quad h = 50 \text{ m}, \quad \delta_r = 10^{-6} \text{ m}. \quad (26)$$

若频道 c 在 s 处无信号且 $\|q - s\| \leq r_{\text{cert}}$ ，则单元内任一点 x 满足

$$\|x - s\| \leq \|x - q\| + \|q - s\| \leq h/\sqrt{2} + r_{\text{cert}} < 1000 \leq \rho_c. \quad (27)$$

该单元若有源，应能接收，与负观测矛盾，因而可整格排除。记 U_c 为尚未排除的单元中心集合， $\mathcal{C}_{\text{pending}}$ 、 $\mathcal{C}_{\text{unheard}}$ 分别为已发现未清除、尚未发现的频道集合，则

$$\mathcal{C}_{\text{pending}} = \emptyset, \quad \forall c \in \mathcal{C}_{\text{unheard}}, U_c = \emptyset \quad (28)$$

构成连续覆盖证书。它保证判据成立时无残留；其成立依赖静止全向源、接收半径下界及无信号观测的可靠性。实际排除只能依据该频道的负观测，经过测站本身不产生排除信息。

为据此选择测站，令 $U = \bigcup_{c \in \mathcal{C}_{\text{unheard}}} U_c$ 。候选站 p 的潜在覆盖收益定义为

$$g(p) = |U \cap B(p, r_{\text{cert}})|. \quad (29)$$

它衡量一次停留可能排除的空间范围；具体频道的检测次数仍需在航路规划中计入。

5.3.2 算法实践

在上述成本与约束下，策略由局部定位逐步扩展为覆盖、清除和检测调度的联合规划。表 5 概括各代机制，以下按其所解决的问题展开。

表 5 五代策略的主要机制与局限

策略	主要机制	局限
field	期望场选点、目标航路排序	未发现频道搜索耗时较长
sweep	利用接收下界系统扫圈	搜索与清除路径重复
tour	覆盖与清除共用航路	固定骨架、点网格判据
adaptive	沿途定位、动态覆盖、连续证书	清除终点偏离规划路点
joint	联合选择航路、测站与检测时机	有限候选的启发式优化

(a) 第一代：期望场选点 field

利用问题二的期望直径场评价补测位置。将各方位对应的场旋转、平移并汇总为 $\bar{E}[D](g)$ ，在当前位置 s 选取使下式最小的候选点 g ：

$$F(g) = \bar{E}[D](g)/1500 + \lambda \|g - s\|/3600, \quad \lambda = 1.5. \quad (30)$$

两项分别反映定位不确定性和移动代价，缺失的场值取 1500 m。目标清除顺序先由最近邻确定，再用 2-opt 交换两条边以缩短航路。该策略侧重已知方位的定位收益，对未发现源仍需额外搜索。

(b) 第二代：系统覆盖 sweep

为提高未发现源的检测机会，原点扫描后，在半径 1100 m 的圆周上设置八个候选测站，利用接收半径下界进行系统搜索。覆盖扩大后再集中清除目标，但两个阶段可能重复经过同一区域，由此产生航路合并的需求。

(c) 第三代：共用航路 tour

以半径 1000 m 的八点环带作为覆盖骨架，将目标 p 插入航路边 (a, b) 时，新增距离为

$$\Delta L(p) = \|a - p\| + \|p - b\| - \|a - b\|; \quad \text{末尾插入时为 } \|a - p\|. \quad (31)$$

按最小增量插入目标，再以保持骨架角向顺序的 2-opt 改善航路。搜索与清除由此合并，但固定测站和离散覆盖判据仍限制搜索完备性。

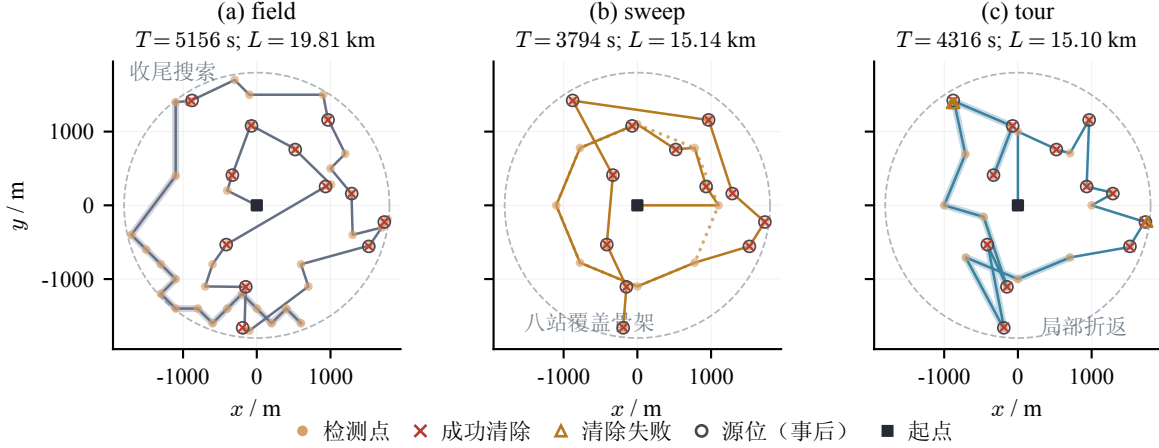


图 7 随机首组的前三代轨迹，均清除 12 源；点线为八站骨架，源位仅作事后标注

图 7 中，sweep 的中心八边形对应八站扫圈，随后转向外围目标；field 则在清除后仍沿边界补充搜索。三者均清除 12 源，但 tour 耗时 4316 s，高于 sweep 的 3794 s，说明航路合并的收益仍受测站和目标分布影响。

(d) 第四代：沿途定位与动态覆盖 adaptive

为摆脱固定骨架，将测站选择与实时观测相结合，并采用式 (28) 判断任务完成。只有一条示向度的目标也纳入清除航路，在接近过程中补测；未发现频道则按式 (29) 的覆盖收益动态选站。

沿途补测首先需要有效的横向基线。原点检测到的频道集合记为 C_{heard} ，在 72 个等分方向中选择

$$\phi^* = \arg \max_{\phi} \sum_{c \in C_{\text{heard}}} \sin^2(\phi - \theta_c), \quad s_1 = 400 u(\phi^*). \quad (32)$$

横向分量越大，测向交会越不易近共线，从而一次移动可改善多个目标的定位。随后，对仅有一条方位的目标，沿估计中心 $\hat{x}_c$ 接近，并沿视线单位法向 n 构造两个补测点：

$$s_{\pm} = s + 0.55(\hat{x}_c - s) \pm \ell n, \quad \ell = \min\{100, \max\{35, 0.25\|\hat{x}_c - s\|\}\}. \quad (33)$$

选取经补测点到估计中心总距离较小的一侧。取得多条方位后继续逼近，必要时侧移；失去信号时向最近正观测位置回退补测。因此，实际清除终点可能偏离原规划路点，其检测收益需要重新评估。

(e) 第五代：航路与检测调度联合优化 joint

针对这一偏移，进一步同时选择覆盖站、航路顺序和检测时机。先排列所有已发现未清除目标，再从候选站中按下式贪心选择并插入：

$$p^* = \arg \max_p \frac{g(p)^{\kappa}}{100 + \max(0, \Delta L(p)) + 30\nu \max(1, |C_{\text{unheard}}|)}. \quad (34)$$

其中 κ 调节覆盖收益偏好, ν 调节检测成本; $30 = 5(5 + 1)$ 将检测和切频的 6 s 折算为行程, 100 m 为代价下限。已有目标点亦可承担检测任务, 其插入增量为零。选中后仅从规划用集合中扣除覆盖单元, 实际排除仍须等待负观测。

adaptive 使用 $(\kappa, \nu) = (2, 0.3)$; **joint** 比较五组权重生成的航路, 具体参数见附录。为计入同站检测不同频道的成本, 对航路 $\pi = (p_1, \dots, p_m)$ 逐站更新各频道的剩余集合 $U_c^{(j)}$, 以

$$\hat{T}(\pi) = L(\pi)/5 + 6 \sum_{j,c} \mathbf{1}\{j \text{ 承担检测任务}, U_c^{(j-1)} \cap B(p_j, r_{\text{cert}}) \neq \emptyset\} \quad (35)$$

比较方案, 其中 $\mathbf{1}$ 为条件成立时取 1、否则取 0 的指示函数。该式近似切频成本并省略后续定位、清除费用, 仅用于规划; 实际总时间按式 (23) 计算。

选站后还可减少绕行。将仅承担检测任务的站称为覆盖站; 先删除冗余站, 再将剩余站 p_0 向航路投影点 z 移动。设 Q 为仅由该站覆盖的必要单元中心, $v = z - p_0$, 则 $p(t) = p_0 + tv$ 必须满足

$$\|p_0 - q + tv\|^2 \leq r_{\text{cert}}^2, \quad q \in Q, \quad 0 \leq t \leq 1. \quad (36)$$

令 $a = \|v\|^2$, $b_q = 2(p_0 - q)^\top v$, $c_q = \|p_0 - q\|^2 - r_{\text{cert}}^2 \leq 0$, 展开为二次不等式。初始点可行, 故 $a > 0$ 时最大允许步长为

$$t_{\max} = \min \left\{ 1, \min_{q \in Q} \frac{-b_q + \sqrt{b_q^2 - 4ac_q}}{2a} \right\}. \quad (37)$$

实际步长留 10^{-8} 余量, 仅接受缩短航路的移动。若 $a = 0$ 则不移动; 若 Q 为空, 则删除覆盖站或取消目标点的检测任务。图 8 将这一约束与连续覆盖判据一并表示。

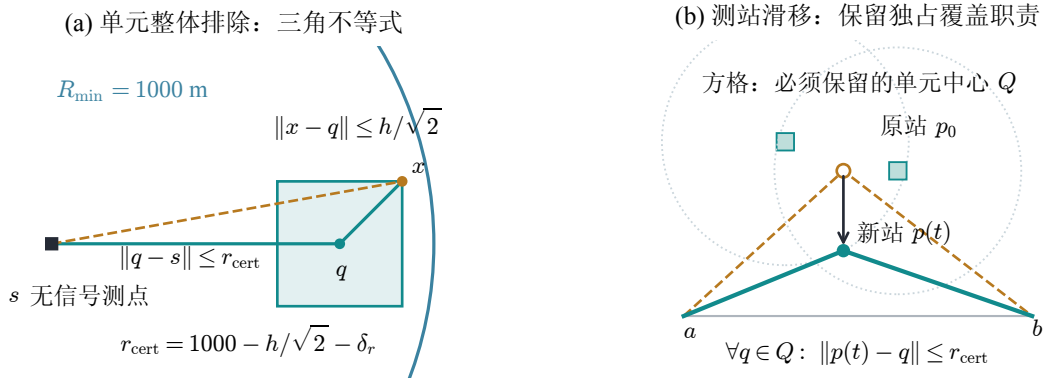


图 8 单元整体排除与保留独占覆盖单元的测站移动

执行时, 每轮仅访问首站, 再按新观测重规划。目标站先定位清除; 若承担覆盖任务, 再以实际终点评估未发现频道的检测收益。覆盖站直接逐频道测量。至少两条方位且 $r_c \leq 400 \text{ m}$ 时允许试探中心, 失败后原地补测并继续定位; 只有 $r_c \leq 20 \text{ m}$ 时清除才有保证。

对沿途可选补测, 还需兼顾接收可能性。沿首条中心射线抽样并结合负观测, 估计接收概率 $\hat{P}_{\text{recv}}$ (附录 4.1)。仅在该概率不低于 0.5, 且移动基线与已有视线的锐角不小

于 25° 时补测；这一近似只筛选可选测量，不删除待清除目标。最近邻、2-opt 和贪心选点均为启发式方法，不保证全局最优。

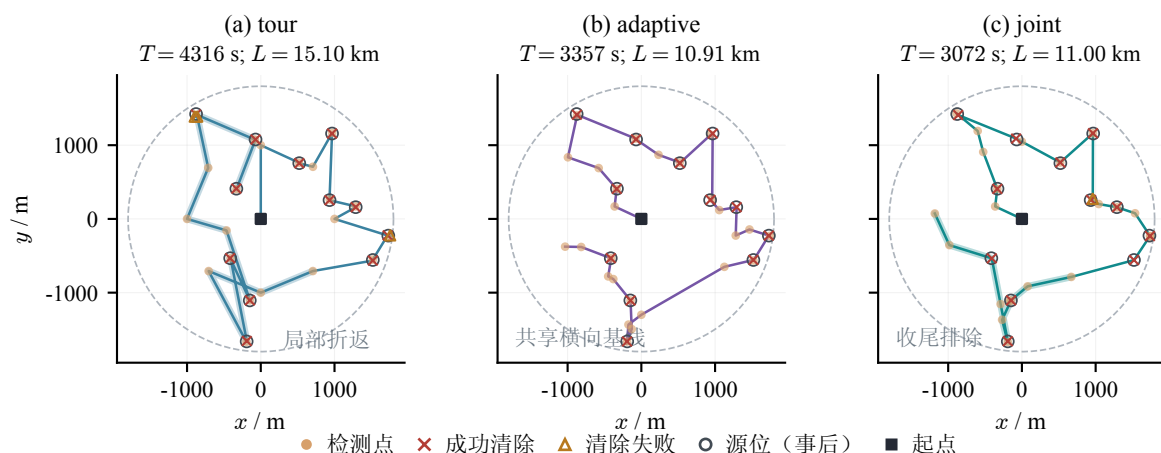


图 9 同组场景下的固定航路、沿途定位与联合调度，均清除 12 源

图 9 中，tour 在南侧多次折返，后两代则沿目标分布推进，并在末端完成剩余区域排除。joint 行程为 11.00 km，略高于 adaptive 的 10.91 km，但检测次数由 186 降至 136，完整时间由 3357 s 降至 3072 s，表明检测调度也能带来独立于行程缩短的收益。

5.3.3 数据分析

(a) 实验设计与总体表现

设置随机、外圈、中心聚集、非平滑固定位置误差及最小接收半径五类场景，分别为 50、20、10、20、20 组。五种策略使用各自默认参数，在相同源位、接收半径和误差场下独立运行，共 600 次；分布与种子见附录。所有轨迹图均取随机集第一组，统计包含未全清实验；P90 表示完整时间的 90% 分位数。

表 6 随机 50 组场景的清除结果与完整时间

策略	清除比例	全清组数	$\bar{T}_{\text{exit}}/\text{s}$	P90/s	每源耗时/s
field	0.995	47/50	5347	5820	422
sweep	0.986	43/50	4186	4610	334
tour	1.000	50/50	4028	4674	317
adaptive	1.000	50/50	3348	3602	264
joint	1.000	50/50	3067	3367	241

清除比例按总源数加权，每源耗时为各组比值的算术平均。全部 120 组中，tour、adaptive 和 joint 均全清，field、sweep 分别全清 114、92 组。

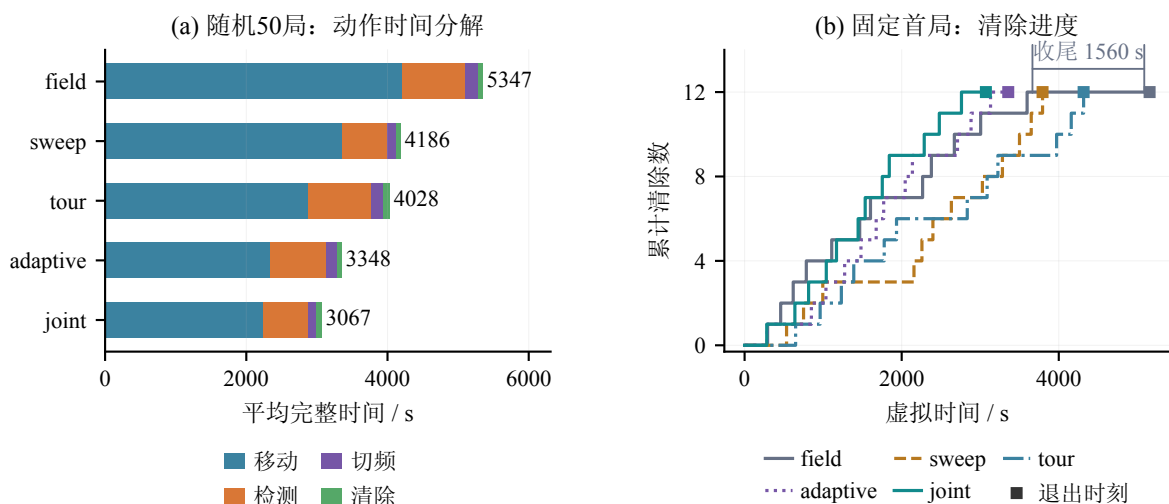


图 10 随机集平均时间分解与第一组场景的累计清除过程

随机集上，joint 较 field 的完整时间均值降低 42.6%，配对 bootstrap 95% 区间为 [41.1, 44.1] %。行程由 21.04 km 降至 11.17 km，检测由 180.5 次降至 127.4 次。

图 10 的首局进度揭示了清除与搜索的时间分配：field 在 786 s 内已清除 4 源，但末次清除后仍搜索 1560 s，因而退出最晚。sweep 前期优先完成扫圈，清除进展较慢，却在 3794 s 完成清除并退出，早于 field；joint 则较早推进清除且缩短收尾，最终最先退出。这说明早期清除速度不能单独代表完整任务效率。

(b) 场景差异与机制消融

图 11 中，joint 在五类场景的平均耗时均最低。最小接收半径组的 sweep 仅 7/20 组全清，说明固定扫圈仍有漏清风险；中心聚集组的 adaptive 耗时 2573.5 s，略高于 tour 的 2529.6 s，表明改进收益具有场景依赖性。

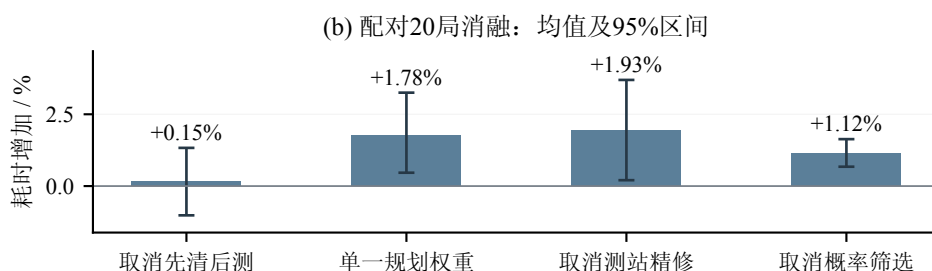
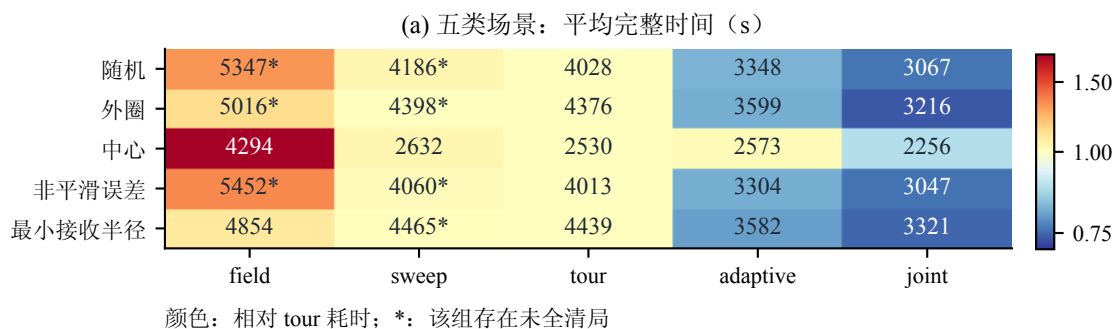


图 11 跨场景时间比较与四项单因素消融的配对区间

在随机前 20 组分别取消清除后扫描、多方案规划、测站精修和概率筛选，新增 80 次实验。相对完整 joint，平均时间分别增加 0.15%、1.78%、1.93%、1.12%，各组均全清。第一项的 95% 配对区间跨零，尚不足以确认其速度收益；其余三项区间为正，支持相应机制的有效性。

由此可见，保守定位与连续覆盖为任务完成提供依据，共用航路和逐频道调度则降低完整成本。上述结论限于所测离线分布，不能推及任意误差场下的快速全清；计算时间和补充统计见附录，正式测试结果见表 7。

表 7 问题三正式测试结果（待补）

测试案例编码	清除干扰源个数	平均定位清除时间	程序运行时间
测试 1 案例编码（待补）	待补	待补	待补
测试 2 案例编码（待补）	待补	待补	待补
测试 3 案例编码（待补）	待补	待补	待补

5.4 问题四：含定向源时的定位与清除模型

5.4.1 模型建立

问题四允许干扰源仅向一个半平面发射。因而，问题三中“近处无信号即可排除该区域”的规则失效：机器狗可能已接近源，却始终位于其背面。为此，将未知发射朝向纳入搜索状态，分别建立发现、定位与退出的证据，再优化取得这些证据的时间成本。

(a) 定向接收模型与任务成本

沿用频道集合 $\mathcal{C}$ 及频道索引 c ；以 $p \in \Omega$ 表示频道 c 的候选源位置，真实位置记为 T_c ，其接收半径为 $\rho_c \in [1000, 1500]$ m，物理发射方向为单位向量 $d(\gamma) = (\cos \gamma, \sin \gamma)$ ，其中 $\gamma \in [0, 2\pi)$ 为发射朝向角。在测站 s 能接收该定向源，当且仅当

$$H(s; p, \rho_c, \gamma) = \mathbf{1}\{\|s - p\| \leq \rho_c, (s - p)^\top d(\gamma) \geq 0\} = 1. \quad (38)$$

全向源去掉第二个条件。因此，无信号意味着“距离超出接收半径”或“测站位于发射背面”，不能单独推出 $\|s - p\| > 1000$ m。发射方向是待排除的隐变量；实现不必先准确辨别全向、定向类型，便可对两者统一搜索。

动作成本沿用 (23)，优化完整退出时间 T_{exit} 。同时记录成功清除数 $N_{\text{clear,ok}}$ 、末次清除时刻 T_{last} 及

$$\bar{t}_{\text{clear}} = T_{\text{last}}/N_{\text{clear,ok}}, \quad \bar{t}_{\text{exit}} = T_{\text{exit}}/N_{\text{clear,ok}}. \quad (39)$$

前者为末次清除口径，后者为完整任务口径，计入末次清除后的残留搜索；本文主要比较完整任务时间，另报告末次清除口径作为补充。 $N_{\text{clear,ok}} = 0$ 时两者不定义，仍保留该局的清除数与总时间。清除比例与时间必须共同报告，避免把遗漏目标带来的提前退出误判为提速。

(b) 从三角格网到任意朝向覆盖

先考虑在三角格网顶点检测。若 p 位于三角形 $v_1 v_2 v_3$ 内，存在 $\lambda_i \geq 0$ 、 $\sum_i \lambda_i = 1$ ，使

$$p = \sum_i \lambda_i v_i \implies \sum_i \lambda_i (v_i - p)^\top d = 0. \quad (40)$$

所以至少一个顶点处于源的发射半平面。对于边长 a_Δ 的等边三角形，内部任一点到任一顶点的距离不超过 a_Δ ，故完整三角剖分配合 $a_\Delta \leq 1000$ m 是一个充分条件。这里 $a_\Delta/\sqrt{3}$ 仅是最近格点的覆盖半径，最近格点未必位于发射正面，不能据此放宽定向覆盖条件。

有限格网还存在边界问题：靶区边缘朝外发射的源可能看不到任何域内测站。于是需要外侧站位，但“补一圈”以及只检查径向朝内、朝外两种情况，仍不足以证明覆盖所有位置与方向。更一般地，对候选源位置 p ，1000 m 内的测站须从各侧围住 p ，即其相对向量的凸包包含原点，才能保证任意发射半平面均有可接收测站。连续区域的计算判断如下。

(c) 连续位置与朝向的无源证书

用闭方格覆盖靶区，保留所有与 Ω 相交的边界格。设格中心为 q 、边长为 h ，格内位置 p 满足 $\|p - q\| \leq \delta_h = h/\sqrt{2}$ 。若某频道在 s 实际无信号，且

$$\|s - q\| + \delta_h \leq 1000, \quad (s - q)^\top d \geq \delta_h, \quad (41)$$

则该格内每个位置均满足

$$\begin{aligned} \|s - p\| &\leq \|s - q\| + \|p - q\| \leq 1000, \\ (s - p)^\top d &= (s - q)^\top d - (p - q)^\top d \geq \delta_h - \|p - q\| \geq 0. \end{aligned} \quad (42)$$

假如该格中存在朝向 d 的源，便应能接收，与无信号矛盾。因此，此次负观测排除的是整个方格内的一段连续朝向，而不只是格中心。

令 $\psi_s = \arg(s - q)$ 、 $\ell_s = \|s - q\|$ 。对满足距离条件且 $\ell_s > \delta_h$ 的测站，第二个不等式等价于排除圆弧

$$I_s(q) = \left[\psi_s - \arccos \frac{\delta_h}{\ell_s}, \psi_s + \arccos \frac{\delta_h}{\ell_s} \right] \pmod{2\pi}. \quad (43)$$

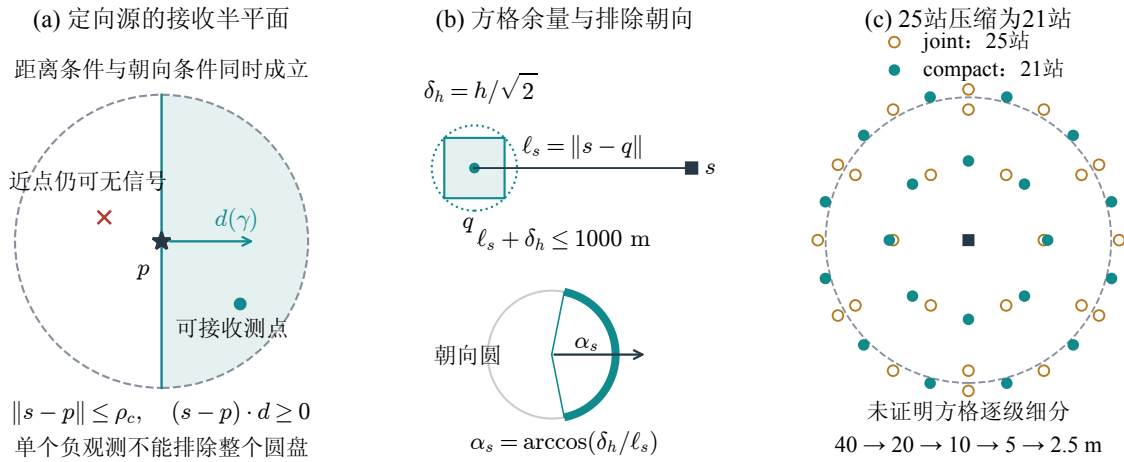
其余测站令 $I_s(q) = \emptyset$ 。记频道 c 的实际无信号测站集合为 Q_c^- ，将跨过零角的圆弧拆分、按起点排序合并，若每个相交方格都有

$$\bigcup_{s \in Q_c^-} I_s(q) = [0, 2\pi), \quad (44)$$

则该频道在靶区内不可能还有任意朝向的定向源；全向源更易接收，也随之排除。最终完成判据为

$$\forall c \in \mathcal{C}, \quad \text{频道 } c \text{ 已成功清除} \quad \vee \quad (\text{无待清正观测且持有连续无源证书}). \quad (45)$$

计划测站只能验证布局可行，最终判据必须使用该频道实际获得的负观测。证书成立说明无残留；预算耗尽而证书未成立时，程序退出并标记未完成。



负观测只排除带余量证明的“位置—朝向”状态；布局认证后仍须逐频道实际检测。

图 12 定向接收、带方格余量的朝向排除与测站布局压缩；方格图为几何示意

5.4.2 算法实践

在上述模型下，算法由格网启发式演进到有证据的联合搜索，再压缩布局、改进估价。表 8 中的三版均保留可运行实现，性能比较使用相同场景重新计算。

表 8 问题四三版策略的递进关系

策略	主要机制	进一步需要解决的问题
grid	三角格网补外环、按环推进、发现后清除	径向覆盖启发式缺少任意朝向证书
joint	25 站连续覆盖、搜索清除共用航路、保守定位	站位较多，纯距离排序忽略检测收益
compact	21 站自适应证书、后验中心、信息代价排序	有限候选局部优化，未证明全局最优

(a) 由覆盖可行性反求紧凑站位

第一版采用边长 1000 m 的域内格网及外环，借鉴问题三按环推进、顺路清除的组织方式。第二版取消径向朝向假设，用 (44) 核验“原点、950 m 六点环、 $950\sqrt{3}$ m 六点错位环、1900 m 十二点环”，共 25 站，并将待扫描站和待清目标合并排序。

在有可靠退出条件后，第三版对同心环的半径、点数和相位进行候选搜索，以站数和开放航程衡量成本，以连续证书约束可行性，最终采用

$$\begin{aligned} \mathcal{S} = \{(0, 0)\} \cup \{998 d(k\pi/4) : k = 0, \dots, 7\} \\ \cup \{1867 d(\pi/12 + k\pi/6) : k = 0, \dots, 11\}. \end{aligned} \quad (46)$$

外环超过靶区半径是为了检测边缘朝外源，题设允许机器狗在靶区外移动。该布局含原点共 21 站，较第二版减少 16%；它是已找到的可行方案，并未证明站数最少。

紧凑布局的几何余量较小，固定粗方格可能无法认证。故从 $h = 40$ m 开始，仅将未证明方格四等分，逐层重复圆弧并集检验；最细允许到 0.625 m，本布局在 $40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 2.5$ m 即全部通过。各层未证明格数为 348, 380, 252, 172, 0；细分后格数增多不意味着覆盖退化，每次接受仍然证明整个方格。这样既减少站数，又保留连续区域的保证。

图 12 展示两版布局的相对位置；完整细分计数列于附录。

(b) 短横向基线与合法负观测约束

发现源后，设 I_c^+ 为有示向度的正观测索引， W_i 为对应角楔， $\mathcal{H}_c$ 为已由合法双侧负观测获得的半平面约束集合，构造

$$P_c = \Omega \cap \bigcap_{i \in I_c^+} (B(S_i, 1500) \cap W_i) \cap \bigcap_{H \in \mathcal{H}_c} H \subseteq \tilde{P}_c. \quad (47)$$

其中 $\tilde{P}_c$ 将圆域各用 32 条外切半平面近似，仍包含真源；角楔半宽为 $(1.005 + 10^{-8})^\circ$ ，其中 0.005° 覆盖读数保留两位小数的舍入误差。它延续问题一的保守定位，但不因单次失去信号而删去距离圆盘。

从最后一个正观测点 s 出发，令示向度单位向量为 u 、法向为 n ，估计中心沿射线的非负投影距离为 $a_{\parallel}$ ；以 t 、 ℓ 分别表示纵向步长和横向偏移，构造

$$s_{\pm} = s + tu \pm \ell n, \quad t = 0.6a_{\parallel}, \quad \ell = \min\{100, \max(22, 0.3 \max(a_{\parallel}, 1))\} \text{ m}. \quad (48)$$

横向基线改善交会角；先测较近一侧，若无信号再测另一侧。必要时将 t, ℓ 同时缩为 $1/2, 1/4$ ，处理越过源或跨出发射半平面的情形。

双侧无信号还可能产生严格的纵向上界。设角误差半宽为 η ，当

$$t \tan \eta \leq \ell, \quad t \geq \ell(\sec \eta + \tan \eta), \quad (49)$$

将任意楔内远端位置写作 $p = s + xu + yn$ ，其中 x, y 为相对于测站的纵、横向局部坐标。对 $x \geq t, |y| \leq x \tan \eta$ ，其与 s 的连线经过横截线段 s_-s_+ 。因为 s 曾接收，至少一个端点与 s 同属该源的发射半平面。又有

$$\begin{aligned} \|p - s_{\pm}\|^2 - \|p - s\|^2 &= t^2 + \ell^2 - 2tx \mp 2\ell y \\ &\leq \ell^2 - t^2 + 2t\ell \tan \eta \leq 0, \end{aligned} \quad (50)$$

故该端点也在源的实际接收半径内。如果两端实际均无信号，便可反证排除 $x \geq t$ ，加入 $u^\top(p - s) \leq t$ 的保守截断。程序留数值余量；条件未满足时不加入该约束。

取得至少两条方位后允许试清除中心；若包围半径不超过 20 m 则清除有保证，否则只是试探，失败后继续补测。最多 8 轮逼近后，对外包多边形采用主轴旋转的方格清除点覆盖，网格间距 $h_{\text{clear}} = 28 \text{ m}$ 。由于

$$\max_p \min_{z \in h_{\text{clear}} \mathbb{Z}^2} \|p - z\| = h_{\text{clear}}/\sqrt{2} = 19.799 \dots < 20 \text{ m}, \quad (51)$$

保留所有距多边形不超过 20 m 的格点，逐个试清即可在有限步骤内命中。此保证依赖真源仍在外包内及预算足够；它利用光学清除与发射朝向无关的题设，而不要求始终收到信号。

(c) 用后验中心改善单方位目标估价

只有一条方位时，几何中心不能体现先前在哪些位置无信号。为改善清除节点的位置估计，在首条中心射线上取 150 个距离点 $p_k = s + t_k u$ ，接收半径取 9 点，发射方向取与正观测相容的 36 点，全向先验权重取 $\pi_o = 0.5$ 。令 $\mathcal{L}_o, \mathcal{L}_d$ 为候选全向、定向状态与实际正负回复相容的指示量，求和变量 ρ 表示候选接收半径，则

$$\begin{aligned} w_k &= t_k \mathbf{1}_{\Omega}(p_k) \left[\frac{\pi_o}{9} \sum_{\rho} \mathcal{L}_o(p_k, \rho) + \frac{1 - \pi_o}{9 \cdot 36} \sum_{\rho, \gamma} \mathcal{L}_d(p_k, \rho, \gamma) \right], \\ \hat{x}_c &= \frac{\sum_k w_k p_k}{\sum_k w_k}, \quad r_c = \max_{v \in \text{vert}(\tilde{P}_c)} \|v - \hat{x}_c\|. \end{aligned} \quad (52)$$

t_k 来自面积先验的极坐标权重。该计算是中心射线上的离散后验近似，仅改变规划中心；完整保守多边形仍被保留，并重算 r_c 。权重和为零时回退到几何中心。因此，先验不准确会影响效率，却不会改变无源证书的成立条件。

图 13 中，双侧负观测只在 (49) 成立时截断远端；中图为固定观测构造的估价示例，右图说明清除网格为何能覆盖保守定位区域。

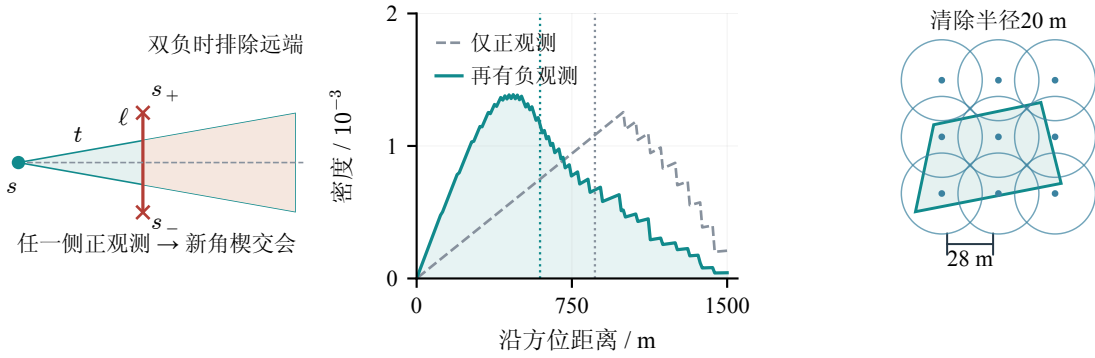
(d) 将检测收益写入开放航路目标

将未访问扫描站与待清目标中心共同组成节点集，从实际位置出发求开放航路 π 。单纯最短距离可能把信息量大的扫描站排到后面，使更多频道在后续各站重复检测。为此，独立生成 1024 个固定参考状态，源位按面积均匀、朝向均匀、接收半径均匀；令

(a) 双侧短基线与双负截断

(b) 单方位后验用于估价

(c) 清除圆盘覆盖兜底

(a) 误差角示意放大10倍；实际 $\eta \approx 1.005^\circ$ 。(b) 负测点为(550,180) m，竖线为均值。

后验均值不缩小保守可行域；兜底依靠清除半径，不依赖信号接收方向。

图 13 双侧补测、单方位后验中心与清除覆盖；左图误差角放大 10 倍，实际半宽约 1.005°

F_k 为前 k 个航路节点及既有扫描点能够接收的参考状态比例，只有扫描节点更新该比例。用

$$J(\pi) = L(\pi) + \frac{30|\mathcal{C}_{\text{unheard}}|}{\omega(F_0)} \sum_{k: \pi_k \text{ 为扫描站}} \omega(F_{k-1}), \quad \omega(F) = 0.35 + 0.65(1 - F) \quad (53)$$

评价方案， $\mathcal{C}_{\text{unheard}}$ 沿用问题三的未发现频道集合， $30 = 5 \times (5 + 1)$ 将一次测量及切频折算为行程。常数 0.35 为扫描代价下限；参考覆盖比例不是该局真实发现概率，该式也不代替逐动作的实际计时。

求解时，以保留上一轮顺序、最小代价插入新节点的航路热启动，再加至多 5 个最近邻起点；每个候选至多作 4 轮 2-opt 片段翻转和单点移位，按完整目标择优。每轮只执行首个节点，随后用新观测和实际清除落点重规划。扫描站检测所有未发现频道，并为满足距离、基线条件的单方位目标补第二条方位；清除点不默认全频道补扫。严格档保留全部必要扫描站，最终仍逐频道核验 (45)。

每轮至多有 20 个剩余扫描站和 20 个目标节点。设节点数为 n ，每种邻域含 $O(n^2)$ 个候选，完整目标重算需 $O(n)$ ，故至多 $K = 6$ 个起点、 $I = 4$ 轮的搜索为 $O(KIn^3)$ ；参考状态的接收掩码可缓存。证书对每个方格合并 m 个测点的圆弧需 $O(m \log m)$ ，只细分未证明格可减少计算量。两者均适合本题有限规模，计算开销另见附录。

5.4.3 数据分析

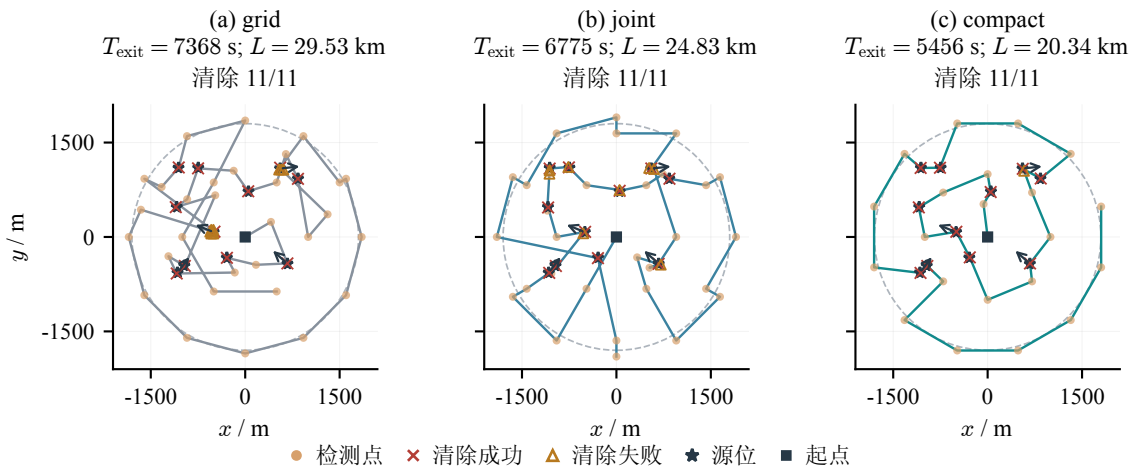
(a) 统一实验与版本比较

采用 30 组随机混合场景，源数为 10 ~ 16，位置按圆域面积均匀，约一半源为随机朝向的定向源；另设最小接收半径、贴边朝外和全向三类压力场景，各 10 组。三版策略在相同源位、接收半径和固定位置误差下独立运行，共 180 次。主集前 10 组增加两项消融和三个速度档，共计 230 次；种子、参数及逐动作审计见附录 F。所有数值均来自此次离线重跑。

表 9 随机混合 30 组的清除结果与完整退出成本

策略	清除比例	全清组数	时间均值/s	P90/s	每源/s	行程/km
grid	100.00%	30/30	8426.1	9392.4	666.7	34.42
joint	100.00%	30/30	6556.9	6944.0	520.1	24.17
compact	100.00%	30/30	5819.1	6131.5	461.3	22.08

每源列为逐局 $T_{\text{exit}}/N_{\text{clear,ok}}$ 的算术平均，清除比例按总源数加权。相较同样具有连续证书的 joint, compact 的随机集完整时间均值降低 11.25%，95% 配对 bootstrap 区间为 [9.14, 13.39]%；其末次清除口径的每源时间为 423.7 s。两个每源口径的差额反映末次清除后继续排除未知频道的成本。



同一预定首局；源位及箭头均为退出后真值，箭头表示可接收半平面的中心方向。

图 14 随机混合第一组的三版实际轨迹；源位及朝向仅在实验后用于绘图

图 14 以同一场景展示轨迹差异。布局压缩、定位估计及航路优化共同改变实际行程与动作次数，不能把整版提速全部归因于减少四个测站。

由图 15 可见，compact 同时减少移动和检测支出。与 joint 相比，随机主集行程从 24.17 km 降至 22.08 km，检测从 275.5 次降至 223.0 次，说明提速并非仅靠缩短几何航程。

(b) 压力场景、机制消融与覆盖取舍

表 10 压力场景下的全清组数及完整时间均值

场景	grid		joint		compact	
	全清	时间/s	全清	时间/s	全清	时间/s
最小接收半径	10/10	8604.9	10/10	7535.2	10/10	6612.2
贴边朝外	4/10	13185.4	10/10	8225.0	10/10	7603.1
全向	10/10	7437.9	10/10	6013.8	10/10	5328.0

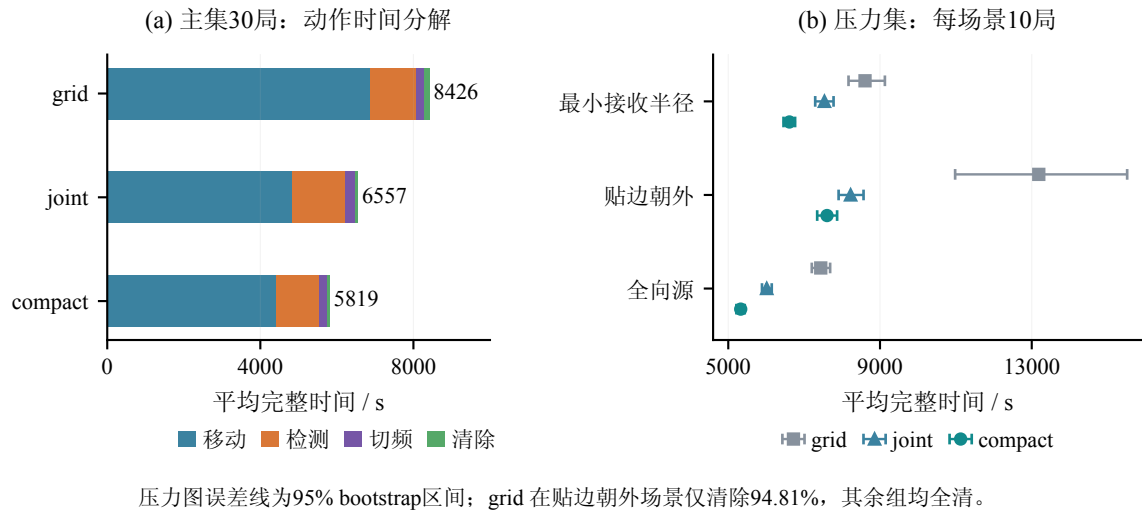


图 15 主集平均动作时间分解与压力场景完整时间的 95% bootstrap 区间

贴边朝外组专门检验域外测站的作用，最小半径组固定接收半径为 1000 m，以检验覆盖余量。全部 60 组中，compact 清除 779/779 个源，60 组取得完整证书。经验全清结果验证了所测场景的表现；任意朝向的可靠退出仍由 (42) 和 (45) 支持，不能由有限样本零失败代替证明。

分别关闭后验中心和信息代价，主集前 10 组的完整时间相对完整 compact 变化为 +7.97% 和 +2.59%。图 16 的配对区间显示，后验中心的收益得到支持，信息代价对应的区间跨零，尚不足以单独确认稳定收益。消融仅改变对应开关，几何证书与清除兜底保持一致。

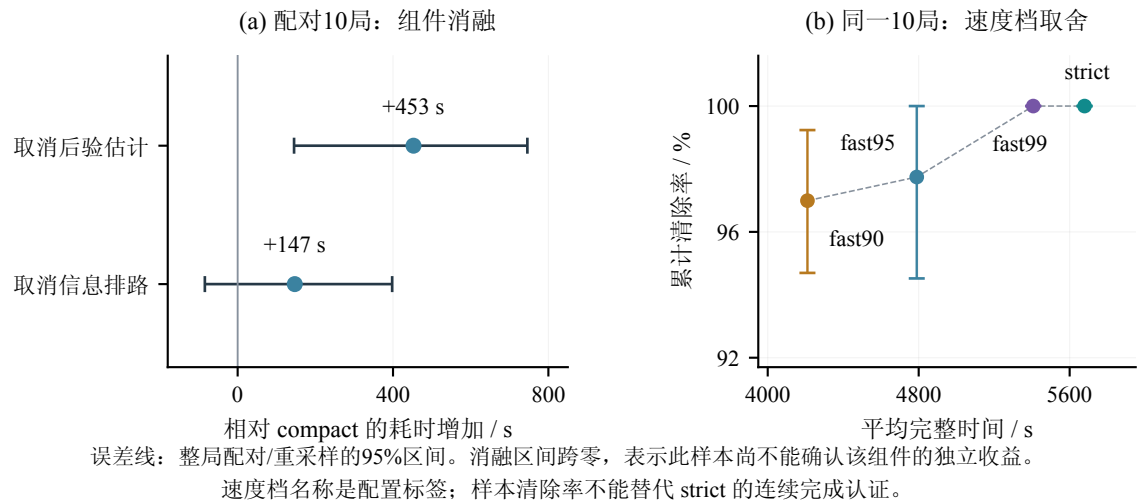


图 16 两项单因素消融的时间变化区间及稀疏速度档的清除率与时间取舍

速度档减少扫描站而提前结束搜索，仍清除已发现目标，却不要求全域证书。fast99/fast95/fast90 是配置名称，不能代表当局真实清除比例或未来全清保证；源分布未知时应选严格档。本节为离线仿真，正式模拟器三次测试数据尚未取得。

六、模型评价与改进

四问形成了“位置约束、检测点选择、联合航路、无残留认证”的递进体系。评价时，既考察定位与退出判据是否可靠，也考察取得这些证据需要付出多少时间；以下结论依据前文推导及统一离线实验。

6.1 参数敏感性与稳健性

误差界影响定位尺度。固定检测点与示向度读数，记误差半宽为 ε 时的角楔交集为 $\mathcal{R}(\varepsilon)$ 。由角楔随误差界增大而扩张，有

$$\varepsilon_1 \leq \varepsilon_2 \implies \mathcal{R}(\varepsilon_1) \subseteq \mathcal{R}(\varepsilon_2), \quad D(\varepsilon_1) \leq D(\varepsilon_2). \quad (54)$$

直径关系针对非空有界交集；误差界继续增大还可能使区域无界。因此，低估误差可能破坏真值包含性，高估误差则可能增加补测成本。问题二的小角近似也表明，远离共线且固定几何与先验时，定位尺度随误差界近似同比增长；近共线布点会放大误差，不能将这一近似推广到全部候选位置。

网格精度影响认证的保守程度。问题三由 (26) 得到有效认证半径 $1000 - h/\sqrt{2} - \delta_r$ ；问题四由 (41) 和 (43) 可见，增大格宽 h 会收紧距离条件并缩短可排除朝向区间。细网格更易取得证书，但增加运算量。Q4 的 21 站布局经未证明方格逐级细分，在 2.5 m 通过核验，说明自适应精度可减小几何离散的保守损失；它不意味着该分辨率对所有布局均足够。

模块收益具有场景依赖性。既有消融中，Q3 取消测站精修后平均增时 1.93%，Q4 关闭后验中心后增时 +7.97%，对应配对区间均为正；但 Q3 清除后扫描、Q4 信息代价的区间跨零（图 (11) 和 (16)）。这支持保留有效机制，也表明不能把每个新增模块都视为稳定提速来源；上述开关实验尚不能确定连续参数的最优取值。

6.2 模型优点

一是定位与完成判据具有可验证依据。问题一直接利用误差界构造可行域，揭示直径圆可能不能包住定位区域；后续采用保守外包及包围半径判断清除条件。问题三、四进一步以实际负观测构造连续覆盖证书，使“已知目标清完”与“区域内无残留”得到明确区分。

二是将定位精度与完整任务成本统一考虑。问题二以期望直径选择补测位置，问题三、四将测站、清除目标及检测调度纳入航路。Q3 随机集最终策略较初代、Q4 紧凑策略较上一严格版的完整时间平均缩短 42.6% 和 11.25%；两项比较各自使用相同场景配对，支持联合规划降低移动与检测开销的作用。

三是具备较完整的数值验证链。几何计算采用独立方法交叉核验，策略比较保留失败场景，并报告完整退出时间、清除比例及配对区间。Q3 最终策略在 120 组场景全清，Q4 最终严格版在 60 组场景清除全部 779 个源并认证；逐动作记录和固定种子使结果能够复核。

6.3 模型局限

首先，**可靠性依赖观测模型成立**。保守定位及无源证书要求源静止、接收半径下界有效、误差有界且接收区内无漏报。遮挡、异常测向或源移动均可能破坏这些条件。证书成立能保证无残留，但尚未证明算法在任意合法场景和有限预算内都能取得证书；离线样本全清也不等于未来零失败风险。

其次，**估价模型存在近似与先验偏差**。问题二的面积先验、小角近似及接收可达性处理限制其适用范围，解析值与数值值在已比较位置吻合，不能替代全域误差界。Q4将单方位位置近似为中心射线，并采用固定全向比例和参考状态估价；源分布变化时，规划中心和检测收益可能失准。问题二的 60 m 阈值是期望性能标准，也不保证每次定位直径均低于该值。

最后，**求解效率与最优性仍有局限**。贪心选点、有限候选及局部航路交换不能保证全局最优，21 站也未证明为最少站数。更强的搜索和更细的认证会增加计算开销，光学网格兜底在定位区域较大时可能耗时较多。本地运行时间未计入真实接口通信、网络抖动等影响，正式测试记录尚待补充。

6.4 改进方向

改进应分别对应上述局限。其一，在位置估价中比较不同空间分布、接收半径及全向比例先验，并对近共线、不同交会角分支和接收截断重新核验几何近似；以信息充分的补测位置降低先验依赖，同时保留保守外包与连续证书。

其二，以同一完整时间目标协调测站选择、补测时机和清除顺序，通过小规模精确求解建立比较基准，再评估局部搜索的改进空间；对覆盖边界采用自适应细分，并设置计算预算，平衡虚拟任务耗时与实际运算时间。

其三，在独立场景中扩大误差场、定向比例和边缘源分布的验证范围，报告超时、漏清及接口失败；若引入漏报或移动源，应重建相应的观测与置信判据，经校准后再用于退出决策，不能直接沿用现有确定性无源证书。

参考文献

- [1] 作者. 无线电测向与定位技术 [M]. 出版社, 年份.
- [2] 作者. 论文题目 [J]. 期刊名, 年份, 卷 (期): 页码.
- [3] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛论文格式规范（2026 年修订稿）.

附录 A 支撑材料文件列表

支撑材料按问题分别存放，包含计算与绘图程序、输入算例和结果数据。以下路径相对于附件根目录 B_locator/。

- 1. Q1/p1_intersection.py: 问题一角楔交会、直径和覆盖判定。
- 2. Q1/p1_experiments.py: 生成五组算例、四组固定种子扫描并写出汇总结果。
- 3. Q1/make_figures.py: 重绘问题一角楔示意、算法流程和覆盖对比三幅插图。
- 4. Q1/cases/: 五组输入 CSV 及真值核验文件; Q1/results/: 逐例结果和汇总 JSON。

5. Q2/pysimulation/src/p2_grid_expectation.py: 问题二数值模拟源程序, 交会四边形直径、加权期望、寻优与候选区域 (附录 C)。
6. Q2/MMAcode/Q2_theory_corrected.wl: 按理论闭式公式进行 200 点中点加权求积的 Mathematica 程序, 导出 15 m 网格上的 $E[D]$ 与最优点。
7. Q2/pysimulation/out/p2_grid/p2_grid_map.csv、p2_grid_report.json: 数值模拟逐点结果与汇总统计 (本文数值来源)。
8. Q2/make_q2_paper_figures.py: 问题二正文插图生成脚本 (由上述两份原始数据出图)。
9. figures/p2-mma-vs-pysim.png: 理论闭式解与数值模拟的并列对比 (图 5)。
10. figures/p2-contour60.png: $E[D] \leq 60$ m 等值线围出的待选区域 (图 6)。
11. figures/p2-wedge-geometry.png: 两探测束交会与定位区域直径构造 (图 4)。
12. Q2/Q2modeling.md 等: 问题二建模手稿、参考热力图与 Mathematica 校验脚本。

问题三附件说明。问题三按评审时的阅读顺序整理为以下四个目录, 论文采用的最终策略为 joint; 五代算法及其默认参数均保留, 便于核对递进比较。

- a. Q3/code/: 五代策略、公共定位与覆盖模块、模拟器; p3_joint.py 为最终算法, SHA256.json 记录执行源码校验值。
- b. Q3/data/: 实验使用的期望场数据和先验场清单。
- c. Q3/results/: 600 局五策略配对实验与 80 局消融的逐局汇总、统计结果及独立审计报告; evidence_map.json 对应论文结论、公式和源码。
- d. Q3/figures/: 五幅正文插图的 PDF、高清 PNG, 以及包含 120 组场景的补充轨迹图册。

Q3/reproduce.py 为统一复现入口; 运行需要 Python 3.12 与 NumPy 2.3, 绘图另需 Matplotlib。以下命令从附件根目录执行:

Listing 1: 问题三的审计与复现入口

```
python -B Q3/reproduce.py --audit-only
python -B Q3/reproduce.py --smoke
python -B Q3/reproduce.py --out /tmp/q3-reproduction
```

第一条仅重放冻结动作并核验成本、真值访问和覆盖声明; 第二条进行离线演练; 第三条在新目录重跑全部实验, 保留失败与漏清局。完整动作日志和研究记录保存在 Q3/research_archive/ 供追溯, 附件的主要阅读与复现方法见 Q3/README.md。附录 E 仅展示最终策略的核心代码。

附录 B 问题一完整源程序

以下代码直接从对应支撑材料文件载入, 各程序按所列文件名保存并保持目录结构即可运行。

2.1 角楔交会、直径与覆盖判定

Listing 2: Q1/p1_intersection.py

```

# -*- coding: utf-8 -*-
"""问题一几何求解。坐标单位 m，示向度为从正东逆时针量取的角度。

命令行: python p1_intersection.py cases/p1_case01.csv
全部实验: python p1_experiments.py
"""
from __future__ import annotations
import csv
import json
import math
import os
from pathlib import Path

import numpy as np

BEARING_ERR = 1.0      # 示向度误差界（度）
ANG_TOL = 1e-9         # 角度判据容差（度）
DIST_TOL = 1e-6        # 半平面判据容差（米）
COVER_TOL = 1e-9       # 覆盖判据容差（米）
CLIP = 1e7             # 裁剪参考实现的外框半边长（米）
DIAM_BRUTE_LIMIT = 128 # 顶点数不超过该值时直径用暴力枚举

def bearing(p, q):
    """p 处指向 q 的方位角（度，[0,360)）"""
    return math.degrees(math.atan2(q[1] - p[1], q[0] - p[0])) % 360.0

def ang_diff(a, b):
    """a-b 归一化到 [-180,180)。"""
    return (a - b + 180.0) % 360.0 - 180.0

def wedge_halfplanes(p, theta, err=BEARING_ERR):
    """角楔 W = {X : |ang(X-p) - theta| <= err} 的两个半平面 a · X + b <= 0"""
    p = np.asarray(p, float)
    tm = math.radians((theta - err) % 360.0)
    tp = math.radians((theta + err) % 360.0)
    dm = np.array([math.cos(tm), math.sin(tm)])
    dp = np.array([math.cos(tp), math.sin(tp)])
    # cross(dm, X-p) >= 0  <=>  a1 · X + b1 <= 0
    a1 = np.array([dm[1], -dm[0]])
    b1 = float(-dm[1] * p[0] + dm[0] * p[1])
    # cross(dp, X-p) <= 0  <=>  a2 · X + b2 <= 0
    a2 = np.array([-dp[1], dp[0]])
    b2 = float(dp[1] * p[0] - dp[0] * p[1])
    return [(a1, b1), (a2, b2)]

def region_is_unbounded(svds, err=BEARING_ERR):
    """非空区域的公共延伸方向；无公共方向时返回 None，复杂度 O(m)。"""
    if len(svds) == 0:
        return None
    ref = float(svds[0])
    lo, hi = -err, err
    for theta in svds[1:]:
        center = ang_diff(theta, ref)
        lo, hi = max(lo, center - err), min(hi, center + err)
        if lo > hi + ANG_TOL:
            return None
    return (ref + (lo + hi) / 2.0) % 360.0

```

```

def _line_intersection(P, u, Q, v):
    """直线  $P + t \cdot u$  与  $Q + s \cdot v$  的交点, 平时返回 None"""
    det = u[0] * (-v[1]) - (-v[0]) * u[1]
    if abs(det) < 1e-14:
        return None
    r = Q - P
    t = (r[0] * (-v[1]) - (-v[0]) * r[1]) / det
    return P + t * u

def convex_hull(P):
    """Andrew monotone chain, 逆时针, 去掉共线点"""
    def half(pts):
        st = []
        for p in pts:
            while len(st) >= 2 and _cross2(st[-1] - st[-2], p - st[-2]) <= 0:
                st.pop()
            st.append(p)
        return st
    P = P[np.lexsort((P[:, 1], P[:, 0]))]
    lo = half(P)
    hi = half(P[::-1])
    return np.array(lo[:-1] + hi[:-1])

def _cross2(u, v):
    return float(u[0] * v[1] - u[1] * v[0])

def _dedup_hull(cand):
    """候选点去重 + 凸包, 返回 (P, "empty"|"degenerate"|"polygon")"""
    if len(cand) == 0:
        return np.zeros((0, 2)), "empty"
    P = np.array(cand, float)
    P = P[np.lexsort((P[:, 1], P[:, 0]))]
    ded = [P[0]]
    for v in P[1:]:
        if np.linalg.norm(v - ded[-1]) > 1e-7:
            ded.append(v)
    P = np.array(ded)
    if len(P) < 3:
        return P, "degenerate"
    H = convex_hull(P)
    if len(H) < 3:
        return H, "degenerate"
    return H, "polygon"

def region_by_vertices(pts, svds, err=BEARING_ERR, tol=DIST_TOL):
    """主算法: 枚举 2m 条边界直线的两两交点, 用全部半平面判据筛选后求凸包。

    R 是凸集, 其任一极点必是两条起作用约束边界直线的交点, 故枚举 + 筛选是精确的;
    半平面判据在处理"边界直线过检测点"的情形时仍良定义, 检测点本身也可作顶点保留。
    """
    pts, svds = np.asarray(pts, float), np.asarray(svds, float)
    if pts.ndim != 2 or pts.shape[1] != 2 or len(pts) != len(svds) or len(pts) == 0:
        raise ValueError("需输入等长、非空的二维检测点和示向度数组")
    if not np.isfinite(pts).all() or not np.isfinite(svds).all() or not 0 < err < 90:
        raise ValueError("坐标和示向度须为有限数, 误差界须在 (0,90) 度内")
    H, lines = [], []

```

```

for p, th in zip(pts, svds):
    H += wedge_halfplanes(p, th, err)
    for s in (-1.0, 1.0):
        a = math.radians((th + s * err) % 360.0)
        lines.append((np.asarray(p, float), np.array([math.cos(a), math.sin(a)])))
cand = []
n = len(lines)
for i in range(n):
    for j in range(i + 1, n):
        X = _line_intersection(lines[i][0], lines[i][1], lines[j][0], lines[j][1])
        if X is None or not np.all(np.isfinite(X)):
            continue
        if all(float(a @ X + b) <= tol for (a, b) in H):
            cand.append(X)
return _dedup_hull(cand)

def clip_halfplane(poly, a, b, tol=DIST_TOL):
    """Sutherland-Hodgman: 保留 {X : a · X + b <= 0} 的部分"""
    if not poly:
        return []
    out = []
    n = len(poly)
    for i in range(n):
        P = poly[i]
        Q = poly[(i + 1) % n]
        fp = float(a @ P + b)
        fq = float(a @ Q + b)
        if fp <= tol:
            out.append(P)
        if (fp < -tol and fq > tol) or (fp > tol and fq < -tol):
            t = fp / (fp - fq)
            out.append(P + t * (Q - P))
    ded = []
    for v in out:
        if not ded or np.linalg.norm(v - ded[-1]) > 1e-9:
            ded.append(v)
    if len(ded) > 1 and np.linalg.norm(ded[0] - ded[-1]) < 1e-9:
        ded.pop()
    return ded

def region_by_clipping(pts, svds, err=BEARING_ERR, box=CLIP, tol=DIST_TOL):
    """有界实例的裁剪校验: 若碰到人为外框则要求增大 box。"""
    poly = [np.array([-box, -box], float), np.array([box, -box], float),
             np.array([box, box], float), np.array([-box, box], float)]
    for p, th in zip(pts, svds):
        for (a, b) in wedge_halfplanes(p, th, err):
            poly = clip_halfplane(poly, a, b, tol)
            if not poly:
                return np.zeros((0, 2)), "empty"
    P = np.array(poly, float)
    if len(P) < 3:
        return P, "degenerate"
    if float(np.max(np.abs(P))) >= box - tol:
        raise ValueError("区域碰到校验外框: 请增大 box 或先检查有界性")
    H = convex_hull(P)
    return (H if len(H) >= 3 else P), "polygon"

def region_from_bearings(pts, svds, err=BEARING_ERR):
    """主入口: 返回 (顶点数组, status), status 取 empty / unbounded / degenerate / bounded。"""

```

```

P, st = region_by_vertices(pts, svds, err)
if st == "empty":
    return P, "empty"
rec = region_is_unbounded(list(svds), err) if len(svds) else None
if rec is not None:
    return P, "unbounded"
if st == "degenerate":
    return P, "degenerate"
return P, "bounded"

def _diameter_brute(P):
    n = len(P)
    best, pair = -1.0, (P[0], P[1])
    for i in range(n - 1):
        d = np.linalg.norm(P[i + 1:] - P[i], axis=1)
        k = int(np.argmax(d))
        if float(d[k]) > best:
            best, pair = float(d[k]), (P[i], P[i + 1 + k])
    return best, pair

def rotating_calipers(P):
    """凸多边形直径（旋转卡壳，O(n)）。P 需为逆时针凸序（允许共线点）。"""
    n = len(P)
    if n < 3:
        return diameter(P, brute_limit=2)
    k = 1
    best, pair = -1.0, (P[0], P[1])
    for i in range(n):
        i2 = (i + 1) % n
        e = P[i2] - P[i]
        while True:
            k2 = (k + 1) % n
            if abs(_cross2(e, P[k2] - P[i])) > abs(_cross2(e, P[k] - P[i])):
                k = k2
            else:
                break
        for j in (k, (k + 1) % n):
            d = float(np.linalg.norm(P[i] - P[j]))
            if d > best:
                best, pair = d, (P[i], P[j])
            d = float(np.linalg.norm(P[i2] - P[j]))
            if d > best:
                best, pair = d, (P[i2], P[j])
    return best, pair

def diameter(P, brute_limit=DIAM_BRUTE_LIMIT):
    """凸多边形直径，返回 (D, (A, B))"""
    n = len(P)
    if n == 0:
        return 0.0, None
    if n == 1:
        return 0.0, (P[0], P[0])
    if n == 2:
        return float(np.linalg.norm(P[0] - P[1])), (P[0], P[1])
    if n <= brute_limit:
        return _diameter_brute(P)
    return rotating_calipers(P)

```

```

def min_enclosing_circle(P):
    """最小包围圆 (center, radius); 顶点数很少, 直接枚举 1/2/3 点候选圆。"""
    P = np.asarray(P, float)
    n = len(P)
    if n == 0:
        return None, 0.0
    if n == 1:
        return P[0].copy(), 0.0
    cand = [(P[i].copy(), 0.0) for i in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            c = 0.5 * (P[i] + P[j])
            cand.append((c, float(np.linalg.norm(P[i] - c))))
    for i in range(n):
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                A, B, C = P[i], P[j], P[k]
                d = 2.0 * (A[0] * (B[1] - C[1]) + B[0] * (C[1] - A[1])
                           + C[0] * (A[1] - B[1]))
                if abs(d) < 1e-12:
                    continue
                aa, bb, cc = float(A @ A), float(B @ B), float(C @ C)
                ux = (aa * (B[1] - C[1]) + bb * (C[1] - A[1]) + cc * (A[1] - B[1])) / d
                uy = (aa * (C[0] - B[0]) + bb * (A[0] - C[0]) + cc * (B[0] - A[0])) / d
                c = np.array([ux, uy])
                cand.append((c, float(np.linalg.norm(A - c))))
    best = None
    for c, r in cand:
        if all(float(np.linalg.norm(P[i] - c)) <= r + 1e-9 for i in range(n)):
            if best is None or r < best[1]:
                best = (c, r)
    return best

def region_area(P):
    n = len(P)
    if n < 3:
        return 0.0
    return abs(0.5 * float(np.sum(P[:, 0] * np.roll(P[:, 1], -1)
                                         - np.roll(P[:, 0], -1) * P[:, 1])))

def analyse(pts, svds, err=BEARING_ERR):
    """给定检测点与示向度, 输出定位区域的全套几何量 (长度量按 4 位小数落盘)。"""
    P, status = region_from_bearings(pts, svds, err)
    base = {"n_bearings": len(pts), "status": status}
    if status == "empty":
        base["reason"] = "各角楔无公共交集, 测量数据与误差界不相容"
        return base
    if status == "unbounded":
        rec = region_is_unbounded(list(svds), err)
        base["reason"] = "非空角楔交集存在公共延伸方向, 直径为无穷大"
        base["recession_direction_deg"] = None if rec is None else round(float(rec), 4)
        base["n_vertices"] = int(len(P))
        base["vertices"] = [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P]
        return base
    if status == "degenerate" or len(P) < 3:
        base["status"] = "degenerate"
        base["reason"] = "公共交集退化为线段或点"
        base["vertices"] = [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P]
        return base

```

```

D, (A, B) = diameter(P)
c = 0.5 * (A + B)
maxr = float(max(np.linalg.norm(v - c) for v in P))
worst = P[int(np.argmax(np.linalg.norm(P - c, axis=1)))]
mc, mr = min_enclosing_circle(P)
base.update({
    "n_vertices": int(len(P)),
    "vertices": [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P],
    "diameter_m": round(D, 4),
    "diameter_endpoints": [[round(float(A[0]), 4), round(float(A[1]), 4)],
                           [round(float(B[0]), 4), round(float(B[1]), 4)]],
    "circle_center": [round(float(c[0]), 4), round(float(c[1]), 4)],
    "circle_radius_m": round(D / 2.0, 4),
    "max_vertex_dist_m": round(maxr, 4),
    "diameter_circle_covers": bool(maxr <= D / 2.0 + COVER_TOL),
    "farthest_vertex": [round(float(x), 4) for x in worst],
    "min_enclosing_center": [round(float(mc[0]), 4), round(float(mc[1]), 4)],
    "min_enclosing_radius_m": round(float(mr), 4),
    "min_circle_over_half_diameter": round(float(mr) / (D / 2.0), 6),
    "area_m2": round(region_area(P), 4),
})
return base

def run_case_csv(path, truth_path=None, err=BEARING_ERR):
    """读单个算例 CSV (列含 x_m,y_m,svd_deg) 与可选真值 JSON, 返回 analyse 结果。"""
    with open(path, encoding="utf-8") as f:
        rows = list(csv.DictReader(f))
    pts = [(float(r["x_m"]), float(r["y_m"])) for r in rows]
    svds = [float(r["svd_deg"]) for r in rows]
    res = analyse(pts, svds, err)
    res["n_bearings"] = len(rows)
    if truth_path and os.path.exists(truth_path):
        with open(truth_path, encoding="utf-8") as f:
            truth = json.load(f)
        G = truth["source_xy"]
        res["truth_xy"] = G
        if "circle_center" in res:
            res["center_error_vs_truth_m"] = round(math.dist(res["circle_center"], G), 3)
        if "min_enclosing_center" in res:
            res["mincircle_error_vs_truth_m"] = round(
                math.dist(res["min_enclosing_center"], G), 3)
        # 真值是否满足全部约束 (等价于在半平面内)
        H = []
        for p, th in zip(pts, svds):
            H += wedge_halfplanes(p, th, err)
        res["truth_inside_region"] = bool(all(a @ np.asarray(G, float) + b <= 1e-6
                                             for (a, b) in H))
    return res

if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("csv", nargs="?", default="cases/pl_case01.csv")
    parser.add_argument("--error", type=float, default=BEARING_ERR)
    args = parser.parse_args()
    path = Path(args.csv)
    if not path.is_absolute() and not path.exists():
        path = Path(__file__).resolve().parent / path
    print(json.dumps(run_case_csv(path, path.with_suffix(".truth.json"), args.error),
                    ensure_ascii=False, indent=2))

```

附录 C 问题二主要源程序

以下列出问题二理论计算的 Mathematica 源程序。程序直接实现式 (18) 的中点加权求积并导出 15 m 网格数据；Python 数值模拟程序保留在支撑材料路径中。

3.1 理论公式的 Mathematica 中点求积

程序运行后在自身目录生成 Q2_expected_diameter_data.csv。

Listing 3: Q2/MMAcode/Q2_theory_corrected.wl

```
(* Q2 theoretical expected diameter; direct midpoint quadrature *)
ClearAll["Global`*"];
baseDir = If[NotebookDirectory[] === $Failed, Directory[], NotebookDirectory[]];
SetDirectory[baseDir];

h = 1500.;
alpha = Pi/90.;
n = 200;
gridA = Range[-500, 1795, 15];
gridB = Range[-1800, 1800, 15];

(* D(x,a,b) from the theoretical derivation *)
d[x_?NumericQ, a_?NumericQ, b_?NumericQ] :=
  If[Abs[b] < 10^-12, Indeterminate,
    alpha/Abs[b] * Sqrt[(x - a)^2 + b^2] *
    Sqrt[(x + Abs[x - a])^2 + b^2]];

(* p(x)=2 x/h^2; n-point midpoint quadrature on [0,h] *)
expectedD[a_?NumericQ, b_?NumericQ] := Module[{xs, w},
  xs = Table[h (k + 0.5)/n, {k, 0, n - 1}];
  w = 2 xs/h^2;
  Total[w (d[#, a, b] & /@ xs)]/Total[w]
];

data = Flatten[
  Table[{N[a], N[b], expectedD[a, b]}, {b, gridB}, {a, gridA}], 1];
cleanData = Select[data, NumericQ[#[[3]]] &];

outFile = FileNameJoin[{baseDir, "Q2_expected_diameter_data.csv"}];
Export[outFile, Prepend[cleanData, {"a", "b", "d_m"}], "CSV"];

k = First@Ordering[cleanData[[All, 3]], 1];
Print["导出完成: ", Length[cleanData], " 行"];
Print["最小值: ", cleanData[[k, 3]], " m, 位置 (a,b)=",
  cleanData[[k, {1, 2}]]];
Print["文件: ", outFile];
```

附录 D 问题三参数、补测概率与复现材料

4.1 补测的中心射线近似

设首个有信号测站为 S_0 ，示向度为 θ_0 。沿 $x(t) = S_0 + t u(\theta_0)$ 在 $5 \sim 1500$ m 上等距取 151 个样本。用极坐标面积元中的 t 为位置权重，并采用接收半径在 $[1000, 1500]$ 上均匀的建模近似。正观测要求 $\rho \geq t$ ，历史负观测 S_j 要求 $\rho < \|x(t) - S_j\|$ ，因此

$$\ell(t) = \max(1000, t), \quad H(t) = \min\{1500, \min_{j \in I^-} \|x(t) - S_j\|\}.$$

I^- 是负观测索引；为空时内层最小值取正无穷。令 $[a]_+ = \max(a, 0)$ ，分母把可能的接收半径区间长度求和，分子进一步要求能在候选站 p 接收：

$$\hat{P}_{\text{recv}}(p) = \frac{\sum_t t \mathbf{1}_{\Omega}(x(t)) [H(t) - \max\{\ell(t), \|x(t) - p\|\}]_+}{\sum_t t \mathbf{1}_{\Omega}(x(t)) [H(t) - \ell(t)]_+}. \quad (55)$$

分母为零时不施加此概率门槛，交由后续几何筛选。它是中心射线上的离散近似，不是完整角楔上的贝叶斯后验。单方位点估计另取 301 个射线样本，剔除靶区外及历史负观测 1000 m 圆盘内样本，作 t 加权平均；该操作不用于缩小保守外包。

4.2 算法参数与实验设置

表 11 问题三算法参数

参数	默认取值与含义
角楔半宽、圆外包	$(1.005 + 10^{-9})^\circ$ ，每圆 32 条切线
清除及近场半径	20 m、5 m；近场仍等待成功清除回复
field 距离系数	$\lambda = 1.5$ ，100 m 行动候选网格
sweep / tour 主圈	(1100 m, 8) / (1000 m, 8)
adaptive 公共基线	400 m，72 个候选方向
沿途逼近	接近比例 0.55，侧移上限 100 m，最多 12 步
连续证书	$h = 50$ m， $r_{\text{cert}} = 964.644660 \dots$ m
joint 覆盖候选	1050、1200、1350、1500 m 四圈，各 72 方向
贪心覆盖 / 规划候选	最多 30 次； (κ, ν) 取 (1, 0.3)、(1, 1)、(2, 0.3)、(2, 1)、(3, 0.3)
测站精修	删除冗余站、保留独占格移动；最多 3 轮
中心试探	至少两条方位， $r_c \leq 400$ m；失败后原地补测
可选补测	概率门槛 0.5、交会基线锐角至少 25°
清除后扫描	after_service_scan='duty'，实际终点更新
小规模精确排序	不超过 3 个路点枚举；其余 NN+2-opt
离线虚拟时钟上限	360000 s；程序墙钟耗时另记，非虚拟成本

各策略使用自身默认配置；最终两代的有效覆盖半径为证书半径。构造性自检不参与性能统计。

4.3 补充统计

随机场景的源数均匀取 10 ~ 16，源位按圆域面积均匀分布；外圈和中心组均固定 12 源，其半径分别在 1300 ~ 1800 m 和 0 ~ 260 m 内均匀取值。接收半径通常在 [1000, 1500] m 内均匀分布，最小接收组固定为 1000 m；非平滑组采用有界位置哈希误差，同地误差固定且不超过 1°。按正文场景顺序编号 $s = 0, \dots, 4$ ，第 i 组种子为 $2026091200 + 100000s + 97i$ ， i 从 0 起计。

下表时间单位为 s，行程为 km；每源列为逐组比值平均，计算列为策略运行的本地实测耗时，不含场景生成与数据分析。

表 12 全部场景的完整成本与清除结果

场景/策略	清除比例	全清	时间均值	P90	每源	行程	检测	计算
random/field	0.995	47/50	5347.3	5820.3	422.2	21.04	180.5	0.040
random/sweep	0.986	43/50	4186.4	4609.7	333.9	16.78	128.1	0.046
random/tour	1.000	50/50	4028.2	4673.7	316.9	14.41	177.1	0.148
random/adaptive	1.000	50/50	3347.9	3601.6	263.6	11.72	156.7	0.228
random/joint	1.000	50/50	3066.7	3366.7	241.1	11.17	127.4	0.707
annulus/field	0.996	19/20	5015.5	5306.7	419.7	18.68	204.3	0.038
annulus/sweep	0.975	15/20	4397.8	5029.2	377.8	16.93	159.2	0.041
annulus/tour	1.000	20/20	4376.3	4774.1	364.7	15.19	210.3	0.125
annulus/adaptive	1.000	20/20	3598.7	3945.9	299.9	12.52	172.8	0.195
annulus/joint	1.000	20/20	3215.8	3455.7	268.0	11.43	145.6	0.472
center/field	1.000	10/10	4293.7	4379.0	357.8	16.05	170.9	0.040
center/sweep	1.000	10/10	2632.3	2772.4	219.4	9.37	116.3	0.055
center/tour	1.000	10/10	2529.6	2759.1	210.8	9.23	100.3	0.110
center/adaptive	1.000	10/10	2573.5	2675.7	214.5	8.23	144.8	0.179
center/joint	1.000	10/10	2256.1	2377.0	188.0	8.01	98.8	0.885
hash/field	0.992	18/20	5451.7	5993.3	425.3	21.53	180.3	0.041
hash/sweep	0.977	17/20	4060.3	4552.1	324.3	16.24	125.0	0.050
hash/tour	1.000	20/20	4012.8	4730.6	308.7	14.73	163.1	0.157
hash/adaptive	1.000	20/20	3303.9	3587.9	256.2	11.67	150.7	0.236
hash/joint	1.000	20/20	3047.0	3261.9	236.6	11.22	121.5	0.734
worstrecv/field	1.000	20/20	4854.4	5105.9	381.0	18.22	191.9	0.035
worstrecv/sweep	0.861	7/20	4465.0	5011.3	410.8	17.16	163.7	0.038
worstrecv/tour	1.000	20/20	4438.8	4987.5	346.5	15.66	204.5	0.163
worstrecv/adaptive	1.000	20/20	3582.2	3811.7	280.9	12.14	182.1	0.215
worstrecv/joint	1.000	20/20	3320.7	3565.5	260.4	11.79	149.9	0.630

四项消融只用随机集前 20 局，完整策略均值 3076.7 s。下表增长率定义为“消融均值/完整策略均值 - 1”，区间是同一组种子重采样的 4000 次配对 bootstrap，未进行多重比较校正；结论限于这组机制比较。

表 13 单因素消融的成本变化及配对区间

消融	时间/s	增长/%	95% 区间/%	行程/km	检测	计算/s
取消清后扫描	3081.2	0.15	[-0.96, 1.32]	11.24	127.4	0.694
单方案规划	3131.3	1.78	[0.48, 3.23]	11.37	131.6	0.237
取消测站精修	3136.1	1.93	[0.17, 3.71]	11.32	134.1	0.314
取消概率门槛	3111.3	1.12	[0.67, 1.66]	11.19	134.2	0.676

表 14 消融子集的清除结果与完整时间分布

策略	清除比例	全清	时间均值/s	P90/s	每源耗时/s
完整 joint	1.000	20/20	3076.7	3424.0	240.3
取消清后扫描	1.000	20/20	3081.2	3333.5	240.4
单方案规划	1.000	20/20	3131.3	3444.7	244.3
取消测站精修	1.000	20/20	3136.1	3415.2	245.2
取消概率门槛	1.000	20/20	3111.3	3461.8	242.9

4.4 文件清单及复现命令

汇总数据保存在 B_locator/Q3/results/；逐动作记录和审计所需的完整输入保存在 B_locator/Q3/research_archive/out/paper_q3/，并附有代码、期望场及场景参数的校验信息。图表从同一组数据生成，完整复现命令如下。

Listing 4: 从仓库根目录复现问题三

```
python -B B_locator/Q3/reproduce.py --audit-only
python -B B_locator/Q3/reproduce.py --out /tmp/q3-reproduction
python B_locator/Q3/research_archive/scripts/make_paper_tables.py
python B_locator/Q3/research_archive/scripts/make_paper_figures.py
python B_locator/Q3/research_archive/scripts/make_paper_trajectory_atlas.py
python B_locator/Q3/research_archive/src/p3_selftest.py --out /tmp/q3-selftest.txt
python B_locator/Q3/research_archive/src/p3_adaptive_checks.py
python B_locator/Q3/research_archive/src/p3_joint_checks.py
bash B_locator/paper/build.sh
```

实验与绘图分别使用 NumPy、Matplotlib，论文以 XeLaTeX 或 Tectonic 编译。

环境与验证说明见附件 Q3/research_archive/PAPER_REPRODUCE.md；全部 120 组五策略轨迹见 Q3/figures/supplementary_trajectories.pdf。

附录 E 问题三最终策略核心代码

问题三的完整可运行代码、输入数据和结果汇总均放在附件 Q3/ 中。为保持附录紧凑，这里只列出最终 joint 策略的核心调度代码；其中调用的公共定位、覆盖和模拟器模块可在附件 code/ 中按同名文件取得，文件校验值见附件的清单。

Listing 5: 问题三最终 joint 策略核心代码

```
"""Q3 joint routing and scan scheduling with continuous coverage certificates.

Only actual replies enter decisions. Probability gates affect optional bearing
measurements, never mandatory clearance or the deterministic completion proof.
"""

from dataclasses import dataclass
import math
import numpy as np
from p3_adaptive import AdaptiveRobot, AdaptiveConfig

@dataclass
class JointConfig(AdaptiveConfig):
    after_service_scan: str = 'duty'
    reachable_filter: bool = True
    polish_small_routes: bool = True
    candidate_radii: tuple = (1050., 1200., 1350., 1500.)
    speculative_radius_m: float = 400.0
    cover_polish: bool = True
    planner_multistart: bool = True
    opportunistic_receive_probability: float = 0.5
    continuous_cover: bool = True
    probe_fail_measure: bool = True
    min_move: float = 0.0

class JointRobot(AdaptiveRobot):
    def __init__(self, arena, cfg=None, base=None, log=None):
        super().__init__(arena, cfg or JointConfig(), base=base, log=log)

    def clear_target(self, ch, rec):
        if self.cfg.speculative_radius_m and rec.n_bearings >= 2 and not rec.cleared:
            br = self.region_info(rec)
            if br and br['radius'] <= self.cfg.speculative_radius_m:
                cost = math.dist(self.pos, br['center']) / 5 + 15
                if self.can_afford(cost):
                    if self.do_clear(*br['center'], ch):
                        return True
                    if self.cfg.probe_fail_measure and self.can_afford(8):
                        resp = self.measure(*self.pos, ch)
                        if resp.get('measure_result') == 'near':
                            return self.do_clear(*self.pos, ch)
        return super().clear_target(ch, rec)

    def nn_2opt(self, seq):
        if self.cfg.polish_small_routes and len(seq) < 4:
            from itertools import permutations
            return list(min(permutations(seq), key=self._route_length)) if seq else []
        return super().nn_2opt(seq)
```

```

def assemble_route(self, dir_sign, targets=None, cursor=None):
    if self.cfg.planner_multistart:
        from dataclasses import replace
        cfg=self.cfg
        proposals=[]
        try:
            for power,weight in ((1.,.3),(1.,1.), (2.,.3),(2.,1.), (3.,.3)):
                self.cfg=replace(cfg,planner_multistart=False,gain_power=power,scan_cost_weight=weight)
                seq,length=self.assemble_route(dir_sign,targets=targets,cursor=cursor)
                proposals.append((self._plan_cost(seq),seq,length))
        finally:
            self.cfg=cfg
            _cost,seq,length=min(proposals,key=lambda q:q[0])
            return seq,length
    seq,length=super().assemble_route(dir_sign,targets=targets,cursor=cursor)
    if not self.cfg.cover_polish or not seq:
        return seq,length
    pts,covers=self._candidate_cover()
    gx,gy,needed=self.uncovered_mask()
    if not needed.any():
        return seq,length
    r2=self.cfg.cover_radius_m**2
    for _ in range(3):
        changed=False
        for w in list(seq):
            if not w.get('cover_duty'):
                continue
            others=np.zeros(len(gx),bool)
            for v in seq:
                if v is not w and v.get('cover_duty'):
                    x,y=v['xy'];others |= (gx-x)**2+(gy-y)**2<=r2
            must=needed&~others
            if not must.any():
                if w['kind']=='cover':
                    seq.remove(w)
                else:
                    w['cover_duty']=False
                    changed=True
                continue
            if w['kind']!='cover':
                continue
            allowed=np.all(covers[:,must],axis=1)
            if not allowed.any():
                continue
            reduced=[v for v in seq if v is not w]
            costs,places=self._insert_cost(pts,reduced)
            costs[~allowed]=np.inf
            j=int(np.argmin(costs))
            old_cost=self._route_length(seq)-self._route_length(reduced)
            if costs[j] < old_cost-1e-5:
                w['xy']=tuple(pts[j]);reduced.insert(int(places[j]),w)
                seq=reduced;changed=True
        if self.cfg.continuous_cover:
            reduced=[v for v in seq if v is not w]
            start=np.asarray(w['xy'],float)
            Q=np.column_stack([gx[must],gy[must]])
            delta=start-Q
            c=np.einsum('ij,ij->i',delta,delta)-r2
            if c.max(initial=-np.inf)>1e-5:
                continue
            chain=np.array([self.pos]+[v['xy'] for v in reduced])
            destinations=[chain[-1]]

```

```

        for a,b in zip(chain,chain[1:]):
            v=b-a;den=v@v
            t=float(np.clip((start-a)*v/max(den,1e-12),0,1))
            destinations.append(a+t*v)
        candidates=[]
        for z in destinations:
            v=z-start;aa=float(v@v)
            if aa<1e-10:continue
            bb=2*delta@v
            roots=(-bb+np.sqrt(np.maximum(0,bb*bb-4*aa*c)))/(2*aa)
            t=max(0.,min(1.,float(roots.min())))-1e-8)
            candidates.append(start+t*v)
        if candidates:
            P=np.asarray(candidates)
            costs,places=self._insert_cost(P,reduced)
            j=int(np.argmin(costs))
            old_cost=self._route_length(seq)-self._route_length(reduced)
            if costs[j]<old_cost-1e-5:
                w['xy']=tuple(P[j]);reduced.insert(int(places[j]),w)
                seq=reduced;changed=True
        seq=self.nn_2opt(seq)
        if not changed:
            break
    return seq,self._route_length(seq)

def _plan_cost(self,seq):
    gx,gy=self.annulus_grid();r2=self.cfg.cover_radius_m**2
    pending={ch:self.unsafe_of(ch).copy() for ch in self.unheard()}
    scans=0
    for w in seq:
        if not w.get('cover_duty'):
            continue
        x,y=w['xy'];mask=(gx-x)**2+(gy-y)**2<=r2
        for ch,m in pending.items():
            if np.any(m&mask):
                scans+=1;pending[ch]=m&-mask
    return self._route_length(seq)/5+6*scans

def _cross_ok(self,rec,x,y):
    if self.cfg.opportunistic_receive_probability>0 and rec.n_bearings==1:
        s=np.array(rec.pts[0]);a=math.radians(rec.svds[0])
        t=np.linspace(5.,1500.,151);u=np.array([math.cos(a),math.sin(a)])
        p=s+t[:,None]*u
        lo=np.maximum(1000.,t);hi=np.full(len(t),1500.)
        for qx,qy,_t,res in rec.meas_log:
            if res=='no_signal':hi=np.minimum(hi,np.linalg.norm(p-(qx,qy),axis=1))
        density=t*(np.linalg.norm(p,axis=1)<=1800.)
        total=np.sum(density*np.maximum(0.,hi-lo))
        heard=np.sum(density*np.maximum(0.,hi-np.maximum(lo,np.linalg.norm(p-(x,y),axis=1))))
        if total>0 and heard/total < self.cfg.opportunistic_receive_probability:
            return False
    if self.cfg.reachable_filter and rec.n_bearings==1:
        br=self.region_info(rec)
        if br:
            poly=br['poly']
            from p3_robot import _dist_point_polygon
            if poly and _dist_point_polygon(np.array([x,y]),np.array(poly))>1500.:
                return False
    return super()._cross_ok(rec,x,y)

def visit(self,wp):
    ch=wp.get('ch')

```

```

if ch is None or self.cfg.after_service_scan=='off':
    return super().visit(wp)
rec=self.recs[ch]
before=self.stats['n_measure']
self.clear_target(ch,rec)
# The route's target was an estimate; perform its scan at the actual
# endpoint after localization, then replan using that measured position.
wp['xy']=tuple(self.pos)
chans=[]
duty=wp.get('cover_duty') or self.cfg.after_service_scan=='all'
if self.cfg.after_service_scan=='replan':
    here=dict(kind='scan_here',ch=None,xy=tuple(self.pos),r=None)
    route,_=self.assemble_route(1.,targets=[here]+self.target_waypoints())
    duty=any(w['kind']=='scan_here' and w.get('cover_duty') for w in route)
for k in self.pending():
    r=self.recs[k]
    if not r.n_bearings and duty and self.adds_coverage(k,*self.pos):
        chans.append(k)
    elif r.n_bearings==1 and self._cross_ok(r,*self.pos):
        chans.append(k)
if chans:
    self.scan_at(*self.pos,chans)
self.note_pos()
# The inherited loop also accepts a successful clear as progress.
return max(1,self.stats['n_measure']-before)

```

附录 F 问题四参数、复现数据与程序

6.1 参数与连续证书细分

表 15 问题四最终严格档的主要参数

参数	取值与作用
严格布局	原点、998 m 八点环、1867 m 十二点环（相位 15° ）
规划用覆盖网格	40 m 方格、48 个带角余量的方向区间
最终无源核验	连续圆弧精确合并；初始 40 m，仅细分未证明格，最细允许 0.625 m
角楔与圆外包	半宽 $(1.005 + 10^{-8})^\circ$ ，每圆 32 条外切半平面
定位横向基线	前进比例 0.6、侧移上限 100 m、下限 22 m；缩放 1, 1/2, 1/4
逼近及清除覆盖	最多 8 轮；小区域阈值 75 m；兜底方格间距 28 m
单方位沿站补测	与上次正测站相距至少 80 m、距估计中心不超过 1450 m
后验中心	距离 150 点、接收半径 9 点、相容朝向 36 点，全向先验 0.5
航路局部搜索	上轮热启动加至多 5 个最近邻起点，至多 4 轮翻转与移位
信息参考状态	1024 个；独立种子 170120；代价下限系数 0.35
虚拟时钟上限	360000 s；预算不足不宣称严格完成

定位圆外包和双侧负观测截断保持真源包含性，后验中心与信息代价只影响动作选择。实际记录的发射方向字段采用“测点指向源”的比较约定；(38) 使用物理发射方向，作图时将该字段转过 180° 。覆盖证明遍历全部连续朝向，因而不依赖字段正反约定。

表 16 21 站布局的连续证书逐层核验

方格边长/m	本层待证明方格	核验后未证明方格
40	6536	348
20	1072	380
10	1112	252
5	624	172
2.5	336	0

后续每层只处理未证明格的子格，并删除与靶区不相交的子格。接受条件一直是整格与全部连续朝向的覆盖。某层未通过只表示未证明，不能仅凭粗网格将其判为真实盲区；源码中的点检仅用于提前拒绝明显不可行布局，不用于接受布局。

6.2 固定实验定义与统计口径

四类场景按随机混合、最小半径、贴边朝外、全向编号 $s = 0, 1, 2, 3$ ，第 i 组种子为 $2026091201 + 1000000s + 79i$ ， i 从零开始。主集 30 组，其余各 10 组。源数在 10 ~ 16 离散均匀抽取；混合集每源以概率 0.5 为定向，方向均匀。最小半径组全部定向且接收半径固定 1000 m；贴边组半径均匀取 1750 ~ 1800 m，接收半径 1000 m，发射半平面朝径向外侧；全向组作为对照。一般源位按圆域面积均匀、接收半径均匀取 1000 ~ 1500 m。同一源、同一测点误差固定，三策略的场景参数完全一致。

后验中心与信息代价各作一项开关消融，速度档取 fast99、fast95、fast90，均预先固定主集前 10 组，复用对应完整严格档作为配对基线。所有 230 次运行均纳入统计；图中代表轨迹固定为主集第一组，不按表现选例。场景和算法源码校验值保存在 Q4/out/paper_q4/manifest.json。

bootstrap 重抽样 10000 次，以整组场景为抽样单位，保留策略间配对关系。版本节省率为 $1 - \overline{T}_{\text{new}}/\overline{T}_{\text{old}}$ ；消融增长率为 $\overline{T}_{\text{ablated}}/\overline{T}_{\text{complete}} - 1$ ，区间均为百分位区间，未作多重比较校正。每源时间先算各局比值再平均，不等同于总时间除以总清除数。退化的零失败样本区间不构成未来全清概率为 100% 的证据。

表 17 单因素消融：相同十组场景中的完整时间变化

设置	时间/s	增长/%	95% 区间/%	全清
关闭后验中心	6131.7	+7.97	[2.51,13.41]	10/10
关闭信息代价	5826.0	+2.59	[-1.47,7.15]	10/10

表 18 稀疏速度档：相同十组场景中的清除结果与两种时间口径

设置	已清/源数	全清	完整时间/s	末清每源/s	退出每源/s
compact	133/133	10/10	5679.0	392.2	436.2
fast99	133/133	10/10	5407.1	395.2	413.2
fast95	130/133	8/10	4789.7	353.1	372.0
fast90	129/133	6/10	4210.5	323.4	330.0

6.3 完整场景统计与计算开销

下表“末清每源”截止末次成功清除，“退出每源”进一步计入残留验证；计算时间为本地进程中策略执行与终止认证的墙钟耗时，不含构造初始化。

表 19 问题四三版策略在全部场景上的补充统计

场景/策略	已清/源数	全清	时间/s	末清每源	退出每源	检测	计算/s
随机混合/grid	388/388	30/30	8426.1	552.5	666.7	237.2	0.09
随机混合/joint	388/388	30/30	6556.9	479.9	520.1	275.5	0.63
随机混合/compact	388/388	30/30	5819.1	423.7	461.3	223.0	4.19
最小接收半径/grid	126/126	10/10	8604.9	639.8	703.9	284.1	0.09
最小接收半径/joint	126/126	10/10	7535.2	602.0	614.8	329.7	0.52
最小接收半径/compact	126/126	10/10	6612.2	509.4	540.1	264.7	2.43

场景/策略	已清/源数	全清	时间/s	未清每源	退出每源	检测	计算/s
贴边朝外/grid	128/135	4/10	13185.4	945.6	1032.6	449.9	0.09
贴边朝外/joint	135/135	10/10	8225.0	599.2	624.9	391.6	0.44
贴边朝外/compact	135/135	10/10	7603.1	571.2	574.7	381.4	1.07
全向/grid	130/130	10/10	7437.9	373.1	580.7	197.7	0.06
全向/joint	130/130	10/10	6013.8	438.6	473.7	257.9	0.50
全向/compact	130/130	10/10	5328.0	383.0	419.9	205.5	3.24

本次采用 Python 3.12.14、NumPy 2.3.5，六个独立进程运行，单局策略执行墙钟最大 7.55 s。并行资源争用和本机环境会改变墙钟时间，不能将其替代正式模拟器的在线程序运行时间。所有 230 份完整轨迹、65229 条动作通过独立落盘复核，移动、检测、切频、成功及失败清除的账本均吻合，拒绝动作数为零。六组已有专项检查共 37 项通过，包括连续证书、极端定位、航路及整策略回归；日志见 Q4/out/paper_q4/checks/。

6.4 文件清单及复现命令

1. Q4/src/: 各版算法与离线场景；Q4/scripts/run_p4.py 为运行转发入口。
 2. Q4/run_paper_q4.py: 固定场景的配对实验、完整动作记录、统计汇总。
 3. Q4/out/paper_q4/runs.jsonl、summary.json: 逐局指标及汇总；traces/: 230 份压缩动作与退出后真值记录。
 4. Q4/out/paper_q4/manifest.json、audit.json: 配置、源位和源码校验信息、独立审计结果。
 5. Q4/make_paper_tables.py、Q4/scripts/make_paper_figures.py: 由同一批冻结结果生成数值与五幅正文图。
 6. Q4/PAPER_REPRODUCE.md: 完整实验定义、运行环境、复现与解释边界。
- 从仓库根目录执行下列命令；新实验写入单独目录，避免覆盖本文冻结数据。

Listing 6: 问题四实验与论文图表的复现命令

```
python B_locator/Q4/run_paper_q4.py --workers 6 --out B_locator/Q4/out/paper_q4_reproduce
python B_locator/Q4/make_paper_tables.py
python B_locator/Q4/scripts/make_paper_figures.py
python B_locator/Q4/scripts/make_paper_source.py
bash B_locator/paper/build.sh
```

附录 G 问题四最终策略核心代码

以下直接载入最终策略的核心源码，展示紧凑站位、联合航路与后验估价的接入方式，以及根据实际负观测逐频道核验完成证书的逻辑。自适应认证、航路优化、后验估计和定位清除分别由 Q4/src/p4_layout_v2.py、p4_route_v2.py、p4_posterior.py 和 p4_homing.py 实现；三版策略、全部依赖及运行入口均保留在 Q4/ 附件中。

Listing 7: Q4/src/p4_compact.py: 最终紧凑策略

```
"""Compact directional survey with adaptively refined continuous proof."""
from dataclasses import dataclass, asdict
import math
import numpy as np
from p4_search import P4SearchRobot, P4SearchConfig, tier_config
from p4_certificate import ring_layout
from p4_layout_v2 import adaptive_complete
from p4_route_v2 import P4RouteV2Mixin
from p4_posterior import P4PosteriorMixin

@dataclass
class P4CompactConfig(P4SearchConfig):
    tier: str = 'strict'
    compact_rings: tuple = ((998., 8, 0.), (1867., 12, 15.))
    stronger_route: bool = True
    route_service_entry: bool = False
    route_v2_rounds: int = 4
    route_v2_starts: int = 5
    route_information: bool = True
    extra_bearings: bool = False
    opportunistic_baseline_m: float = 300.
    defer_single: bool = False
    defer_distance_m: float = 1100.
    defer_angle_deg: float = 70.
    posterior_estimate: bool = True
    posterior_omni_prior: float = .5
    rotate_layout: bool = False
    rotation_candidates: int = 14

class P4CompactRobot(P4PosteriorMixin, P4RouteV2Mixin, P4SearchRobot):
    def __init__(self, arena, cfg=None, **kwargs):
        cfg = cfg or P4CompactConfig()
        if cfg.certify and not adaptive_complete(ring_layout(cfg.compact_rings)):
            raise ValueError('Compact layout lacks a continuous certificate')
        super().__init__(arena, cfg, **kwargs)

    def _build_stations(self):
        return ring_layout(self.cfg.compact_rings, False)

    def _opening_baseline(self):
        if self.cfg.rotate_layout and self.heard_pending():
            base = np.array(self.stations)
            targets = [dict(kind='clear', ch=ch, xy=tuple(self.estimate(ch)[0])) for ch in self.heard_pending()]
            best = None
            # The certified arena is a disk, so a rigid rotation preserves
            # its geometric validity. Pick phase using observed targets only.
            symmetry = math.gcd(*(int(n) for _, n, _ in self.cfg.compact_rings))
            for phi in np.arange(self.cfg.rotation_candidates * 2 * math.pi / (symmetry * self.cfg.rotation_candidates)):
                mat = np.array([[math.cos(phi), -math.sin(phi)], [math.sin(phi), math.cos(phi)]])
```

```

        pts=base@mat.T
        items=[dict(kind='station',si=i,xy=tuple(p)) for i,p in enumerate(pts)]+targets
        self._route_v2_previous=[]
        seq=self._ordered(items)
        cost=self._length(seq,self.pos)
        if best is None or cost<best[0]:best=(cost,pts)
        self.stations=[tuple(p) for p in best[1]]
        self._station_bits=[np.packbits(self.certificate.station_mask(p)) for p in self.stations]
        self._route_v2_previous=[]
        return super()._opening_baseline()

def _ordered(self,items):
    if self.cfg.stronger_route:return P4RouteV2Mixin._ordered(self,items)
    return P4SearchRobot._ordered(self,items)

def _route(self):
    seq=super()._route()
    if not self.cfg.defer_single:return seq
    scans=[w for w in seq if w['kind']=='station']
    if not scans:return seq
    keep=[]
    for w in seq:
        if w['kind']!='clear' or self.recs[w['ch']].n_bearings!=1:
            keep.append(w);continue
        rec=self.recs[w['ch']];c=np.array(w['xy']);old=np.array(rec.pts[-1])-c
        can_defer=False
        for scan in scans:
            q=np.array(scan['xy']);v=q-c;dist=np.linalg.norm(v)
            if dist>self.cfg.defer_distance_m or math.dist(q,rec.pts[-1])<300:continue
            angle=math.degrees(math.acos(float(np.clip((old@v)/max(1e-9,np.linalg.norm(old)*dist),-1,1))))
            if angle<self.cfg.defer_angle_deg:can_defer=True;break
        if not can_defer:keep.append(w)
    return self._ordered(keep)

def _certify_remaining(self):
    cache={}
    for rec in self.recs.values():
        if rec.cleared:continue
        if rec.n_bearings or rec.near_hits:return False
        points=tuple(sorted(set(tuple(row[:2]) for row in rec.meas_log if row[3]!='no_signal'))))
        if points not in cache:cache[points]=adaptive_complete(points)
        if not cache[points]:return False
    return True

def _scan_actual(self,p,extra=True):
    n=super()._scan_actual(p,extra)
    if self.cfg.extra_bearings and extra:
        for ch in self.heard_pending():
            rec=self.recs[ch]
            if rec.n_bearings<2:continue
            c,rad,_=self.estimate(ch)
            if rad<20 or c is None or math.dist(c,p)>1000:continue
            if math.dist(rec.pts[-1],p)<self.cfg.opportunistic_baseline_m:continue
            self.scan_at(*p,[ch])
    return n

def compact_config(tier='strict'):
    """Second-round presets; fast tier labels are empirical targets only."""
    base=tier_config(tier)
    options=asdict(base)
    options.update(stronger_route=True,posterior_estimate=True,route_information=True)

```

```
if tier=='strict':
    options['compact_rings']=((998.,8,0.), (1867.,12,15.))
else:
    options['compact_rings']=tuple((r,n,0. if k%2==0 else 180./n)
                                   for k,(r,n) in enumerate(base.search_rings))
return P4CompactConfig(**options)
```